



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación 1

Informe Corregido — Semana 5

**Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje
de Gran Escala (LLM).**

Integridad metodológica, RAG por modelo y sistema desplegable

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Junio 2026

Resumen

En este informe corrijo y completo mi trabajo de la Semana 5. Primero resuelvo un problema de **integridad de datos** (una fuga entre entrenamiento y prueba que detecté en mi propia evaluación) y reporto el número honesto de mi mejor modelo. Luego consolido los requerimientos del asesor (representación vectorial, BLEU, distribución de datos, tiempos, estadísticas completas del corpus y la definición de calibración). Re-evalúo **RAG aplicado a todos los modelos** sobre mi conjunto de prueba limpio, corrigiendo los fallos de mi implementación previa. Finalmente construyo y pruebo un **sistema desplegable** que recibe una tesis y reporta el porcentaje de paráfrasis por sección. Mi mensaje central es honesto: ningún aumento de datos sintéticos supera a los 144 pares humanos con los que entrené BETO v3.

Índice

1. Metodología general	3
1.1. Lógica e implementación de cada fase	4
2. Profundización técnica	6
2.1. Representación vectorial: los conceptos	6
2.2. Cómo funciona cada modelo	6
2.3. BLEU en profundidad	7
2.4. Latencia y tiempos de ejecución	8
2.5. RAG en profundidad	8
2.6. El sistema desplegable, paso a paso	8
3. Fase 0 — Integridad de datos (lo primero)	10
3.1. El problema	10
3.2. La corrección	10
3.3. Resultado honesto	10
4. Fase 1 — Requerimientos del asesor consolidados	11
4.1. Representación vectorial por modelo (R1)	11
4.2. BLEU boxplots (R2)	11
4.3. Distribución de los datos (R3)	12
4.4. Estadísticas completas del corpus (R8)	12
4.5. Tiempos y latencia (R5)	13
5. Calibración frente a entrenamiento	13
6. Los conjuntos de datos y sus distribuciones	14
7. Fase 2 — RAG aplicado a TODOS los modelos	15
7.1. Corrección de mi implementación previa	15
7.2. Resultados por modelo (test limpio, 38 pares)	15
8. ¿Hay que meter toda la base de datos al modelo?	16
9. El sistema desplegado de David Luna	16

10. Nuestro sistema: tesis completa → % de paráfrasis por sección	16
10.1. Alcance del bagaje y versión de corpus completo	17
10.2. Resultados del sistema sobre tesis de prueba	17
10.3. Validación con caso positivo conocido	18
11. Interfaz del sistema y demostración	19
11.1. Modo 1 — comparar dos textos	19
11.2. Modo 2 — buscar en el corpus (RAG)	20
11.3. Datos enriquecidos de cada coincidencia	21
12. Comparación con el paradigma de Luna	22
12.1. Cómo lo implementé	22
12.2. Resultados (mismo test, mismas métricas)	22
12.3. Intervalos de confianza (bootstrap)	22
12.4. Interpretación	23
13. Fase 4 — El ciclo sintético como HALLAZGO, no como fracaso	24
14. Puntos a discutir con el asesor	25
15. Análisis profundo: la evolución de BETO y el gold estándar	26
15.1. Tabla comparativa de las siete versiones	26
15.2. Por qué salieron esos valores de F1	26
15.3. Por qué un modelo dio $F1 = 1.000$	26
15.4. Por qué surgió la fuga de datos y cómo la detecté	27
15.5. ¿Cómo mejorar el gold estándar? ¿Balancearlo conviene?	27
15.6. ¿El gold estándar sirve solo si es humano? ¿Y si lo hace una IA?	28
15.7. Propuesta: active learning + comité de IA con árbitro humano	28
15.8. Síntesis y próxima dirección	29
16. Análisis complementarios	31
16.1. BLEU como baseline léxico (diagramas de caja y dispersión)	31
16.2. Catálogo de representación vectorial (los 8 modelos)	31
16.3. Aumento de datos: back-translation	31
16.4. Limitaciones	32
16.5. Trabajo futuro	32
17. Marco teórico ampliado: análisis de los artículos científicos	34
17.1. Nadaş et al. (2025): generación de datos sintéticos con LLMs	34
17.2. Zhang et al. (2019): PAWS y los negativos difíciles	35
17.3. Yang et al. (2019): PAWS-X y la evaluación multilingüe	36
17.4. Jayawardena y Yapa (2024): ParaFusion y la calidad del corpus LLM	37
17.5. Lau y Zubiaga (2024): paráfrasis humanas en detección de texto LLM	38
17.6. Ortiz Barajas (2023): Sentence-CROBI y la arquitectura Transformer para detección de paráfrasis	39
17.7. Síntesis transversal: convergencias y divergencias	40
18. Conclusión	42

1. Metodología general

Organicé esta semana como una cadena de cinco fases, donde cada una depende de la anterior. La figura 1 resume el flujo completo: primero se garantiza la integridad de los datos, luego se consolidan los requerimientos del asesor, se re-evalúa RAG, se construye el sistema desplegable y, finalmente, se encuadra el hallazgo central.

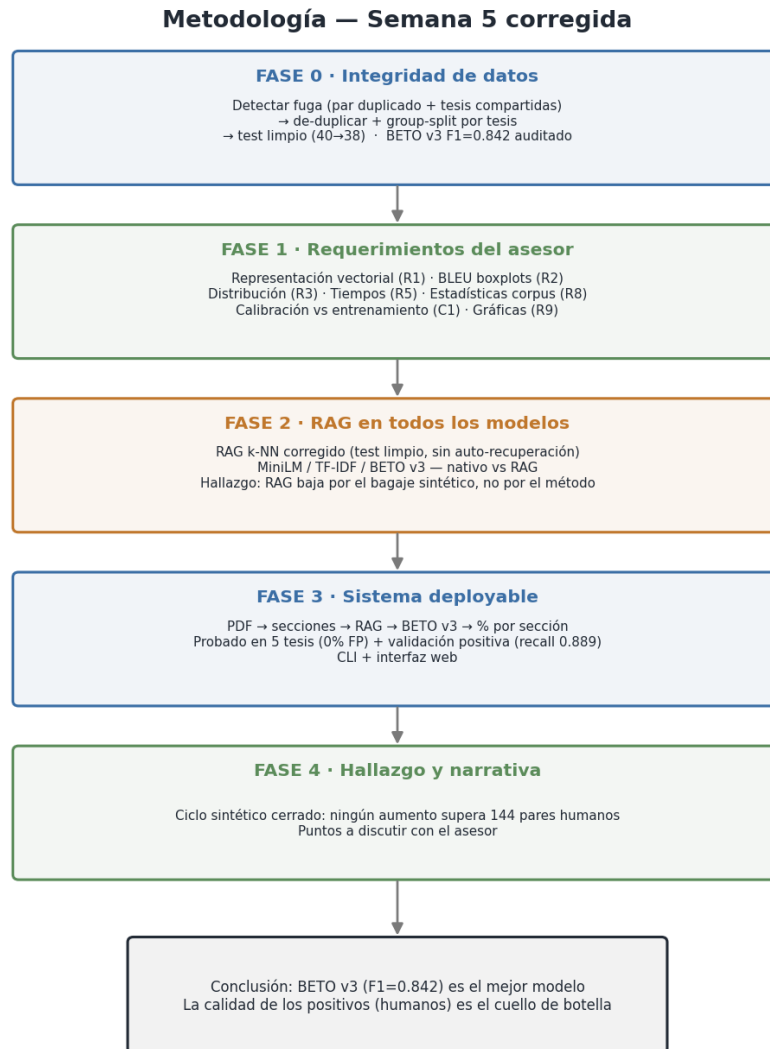


Figura 1: Flujo metodológico de la Semana 5 corregida. Cada fase produce una salida que alimenta a la siguiente; la conclusión es que el mejor modelo (BETO v3) está limitado por la calidad de los positivos, no por la cantidad de datos.

1.1. Lógica e implementación de cada fase

A continuación se explica *qué* se hizo, *cómo* se implementó y *por qué*, fase por fase. Todos los scripts están en `semana_5_correjada/implementaciones/` y comparten el mismo intérprete y semilla (`seed=42`) para reproducibilidad.

Fase 0 — Integridad de datos (`F0_integridad_datos.py`, `F0_evaluar_modelos_limpio.py`).

Mi lógica: antes de reportar cualquier métrica quise asegurarme de que mi conjunto de prueba no contuviera nada que el modelo hubiera visto en entrenamiento, porque eso inflaría los resultados. *Cómo lo implementé:* primero normalizo cada texto (minúsculas y espacios colapsados) y calculo un *hash* del par sin orden —de modo que (a, b) y (b, a) cuenten igual— para detectar pares idénticos entre `train` y `test`. Luego, como cada par lleva en su identificador la tesis de la que proviene, extraigo esas tesis y aplico un *group-split*: elimino del test todo par cuya tesis ya aparezca en entrenamiento, para que ningún fragmento de una misma tesis quede de los dos lados. Con eso el test pasó de 40 a 38 pares. Finalmente re-evalué los cinco modelos BETO (v3–v7) sobre el test limpio con umbral fijo $\theta = 0,50$ y *sin* re-entrenar, para obtener el número honesto del modelo que ya tenía. El detalle clave: los 2 pares que quité eran negativos bien clasificados, así que el F1 no cambió —lo que confirma que el 0.842 es real.

Fase 1 — Requerimientos del asesor (34a, 33, 34b, 35, `F1_estadisticas_corpus.py`). *Mi*

lógica: el asesor pidió “entender lo que hace el modelo”, así que mi objetivo fue dar visibilidad y soporte cuantitativo a cada cosa. *Cómo lo implementé:* catalogué la representación vectorial de cada modelo (dimensión, tokenizador, pooling y formato); calculé BLEU-1/2/4/avg con NLTK y los grafiqué como diagramas de caja y de dispersión por clase, para ver el solapamiento entre paráfrasis y no-paráfrasis; medí la latencia por par de cada modelo; y recalculé las estadísticas del corpus que faltaban (palabras por fragmento y distribución por sección), que antes solo tenían el conteo bruto. Además definí formalmente la diferencia entre *calibración* (elegir el umbral θ , sin tocar el modelo) y *entrenamiento* (modificar los pesos), porque en la sesión anterior los había confundido.

Fase 2 — RAG en todos los modelos (`F2_rag_todos_modelos.py`). *Mi lógica:* el asesor me

pidió aplicar RAG a todos los modelos y no concluir “RAG no sirve” sin sustento. Al revisar mi implementación anterior encontré dos fallos graves: evaluaba sobre el propio corpus (se recuperaba a sí misma) y las variantes “naive” y “stratified” eran el mismo código. *Cómo lo implementé:* rehíce el esquema. Represento cada par por su *vector de relación* $|\text{emb}_1 - \text{emb}_2|$ (la diferencia entre los embeddings de los dos textos): si es pequeña, los textos son cercanos. Para cada par del test recupero del bagaje los k vecinos más parecidos en ese espacio, en dos variantes —*naive* (los k más cercanos) y *stratified* ($k/2$ positivos + $k/2$ negativos)— y voto pesando por similitud. Comparo todo contra la predicción nativa de MiniLM, TF-IDF y BETO v3 sobre el test limpio, asegurándome de que el gold no esté en el bagaje (sin auto-recuperación).

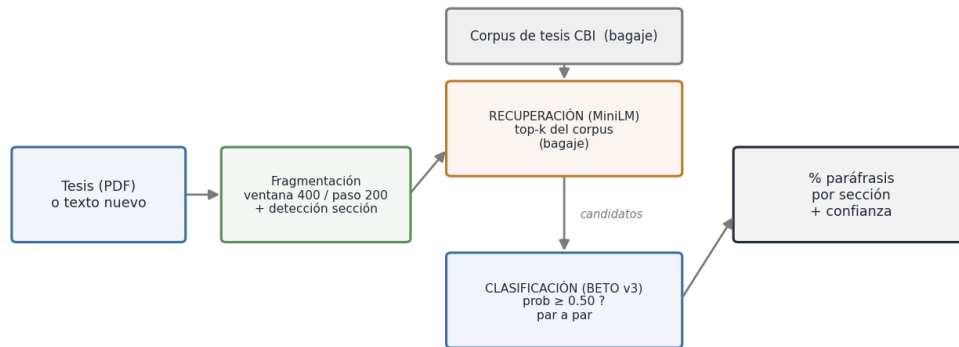
Fase 3 — Sistema desplegable (`F3_sistema_deployable.py`, `F3b_validacion_caso_positivo.py`, **carpeta sistema/**). *Mi lógica:* cumplir la “meta grande” del asesor —meter una tesis y obtener

el % de paráfrasis por sección— usando el corpus como bagaje vía RAG, sin meter todo el corpus al modelo. *Cómo lo implementé:* del PDF extraigo el texto con `pdftotext`, lo fragmento con una ventana deslizante de 400 caracteres y paso de 200 (para no cortar ideas), y detecto la sección de cada fragmento con expresiones regulares. Por cada fragmento recupero los top- k más similares del corpus con MiniLM (rápido) y los clasifico con BETO v3 (preciso); luego agrego el % por sección y, gracias al índice de tesis, indico el título y autor de la tesis con la que coincidió. Todo se apoya

en un núcleo (`deployable_core.py`) que carga los modelos una sola vez y que reutilizan tanto mi línea de comandos como la interfaz web. Para comprobar que el sistema *sí* detecta, hice además una validación con un caso positivo conocido.

Fase 4 — Hallazgo y narrativa. *Mi lógica:* no esconder el resultado “negativo” del ciclo sintético, sino convertirlo en una contribución. *Cómo lo implementé:* consolidé la comparación de las cinco versiones de BETO sobre el test limpio y mostré que lo que determina el desempeño es la *calidad* de los positivos, no su cantidad: a más datos sintéticos, más falsos positivos. De ahí saco mi conclusión y los puntos que quiero discutir con el asesor.

Arquitectura del sistema deployable (RAG + BETO v3)



El corpus NO entra al modelo: la recuperación filtra candidatos y el clasificador decide par a par.

Figura 2: Arquitectura del sistema desplegable. El corpus es el bagaje: la recuperación (MiniLM) filtra candidatos y el clasificador (BETO v3) decide par a par. El corpus completo nunca entra al modelo.

2. Profundización técnica

En esta sección explico en detalle los conceptos y mecanismos detrás de cada componente de mi proyecto: cómo representa el texto cada modelo, qué significan términos como *dimensión*, *pooling* o *formato*, cómo funciona BLEU, qué mide la latencia, y la lógica interna de RAG y de mi sistema desplegable.

2.1. Representación vectorial: los conceptos

Un modelo no «entiende» texto directamente: primero lo convierte en **vectores de números** (llamados *embeddings*). La comparación entre textos se vuelve entonces una operación matemática entre vectores. Los términos de la tabla de modelos significan lo siguiente:

Tokenización. Antes del vector, el texto se parte en *tokens* (unidades). Métodos como *WordPiece* (BERT/BETO) o *subword* dividen palabras raras en piezas (p.ej. «oxidación» → «oxida», «#ción»), para no quedarse sin vocabulario ante palabras nuevas.

Dimensión (*dim*). Es el **número de componentes del vector**. Un embedding de 384-dim es un punto en un espacio de 384 ejes; uno de 768-dim, en 768 ejes. Más dimensiones permiten codificar más matices semánticos, pero cuestan más memoria y cómputo. No «más es mejor» automáticamente: depende de cómo se entrenó.

Pooling. Un modelo produce un vector *por cada token*. Para obtener un único vector que represente toda la oración hay que *agregarlos*:

- **Mean pooling:** promedia los vectores de todos los tokens. Es lo que usan los *bi-encoders* (MiniLM, MPNet, E5). Captura el «promedio semántico» de la frase.
- **Token [CLS]:** BERT/BETO añaden un token especial al inicio; su vector final se usa como resumen de la oración, y es el que alimenta al clasificador en el *fine-tuning*.

Formato. Describe la forma del vector:

- **Denso:** casi todos los componentes son distintos de cero (embeddings neuronales). Captura semántica.
- **Disperso (*sparse*):** la mayoría de componentes son cero (TF-IDF). Cada dimensión corresponde a una palabra del vocabulario.

Bi-encoder vs. cross-encoder. Es la distinción más importante:

- Un **bi-encoder** codifica cada texto *por separado* y luego compara los dos vectores (coseno). Rápido: los vectores se pueden pre-calcular. MiniLM, MPNet, E5, SBERT son bi-encoders.
- Un **cross-encoder** mete *los dos textos juntos* al modelo y produce directamente una decisión. Más preciso (ve la interacción palabra a palabra) pero más lento (no se puede pre-calcular). **BETO v3 en este proyecto es un cross-encoder.**

2.2. Cómo funciona cada modelo

MiniLM-L12-v2 (384-dim, bi-encoder). Versión *destilada* (comprimida) de un BERT, entrenada sobre 1 000 millones de pares de oraciones. Produce embeddings de oración por mean pooling; la similitud se mide por coseno. Es el **recuperador** del sistema: rápido y suficiente para filtrar.

MPNet (768-dim, bi-encoder). Combina el enmascaramiento de BERT con la permutación de XLNet en el pre-entrenamiento, lo que suele dar embeddings de oración de mayor calidad que MiniLM, a costa de más parámetros.

E5-large (1024-dim, bi-encoder). Entrenado con *contrastive learning*: aprende acercando pares relacionados y alejando los no relacionados. Usa prefijos (**query:** / **passage:**) para distinguir el rol del texto. Embeddings grandes y de alta calidad, pero pesado.

SBERT-es (768-dim, bi-encoder). Sentence-BERT multilingüe: un BERT afinado específicamente para que el coseno entre embeddings refleje similitud semántica de oraciones.

BETO (768-dim, cross-encoder en v3). BERT-base en español (110M parámetros), pre-entrenado en Wikipedia y Common Crawl en español. En modo clasificación recibe el par de textos, usa el vector [CLS] y una capa lineal final produce la probabilidad de paráfrasis.

Llama 8B / 70B (decoder-only). Modelos generativos grandes. No producen un embedding de oración «de fábrica»; se usan vía *In-Context Learning* (se les da el par en el prompt y responden SÍ/NO) o tomando los *estados ocultos* de su última capa (miles de dimensiones).

TF-IDF (300–5000-dim, disperso). No es neuronal. Cada dimensión es una palabra; el valor es la frecuencia del término ponderada por su rareza en el corpus (*term frequency* $\times$ *inverse document frequency*). Mide superposición de palabras, no significado.

SVM + embeddings. Un clasificador clásico (máquina de vectores de soporte) que traza una frontera entre clases sobre los embeddings. Se probó con kernels lineal, RBF y polinomial; el lineal fue el mejor, indicando que los embeddings ya son separables sin transformaciones complejas.

2.3. BLEU en profundidad

BLEU (*Bilingual Evaluation Understudy*) mide cuánto se **solapan las palabras** entre dos textos. Originalmente sirve para evaluar traducción automática; aquí lo uso como **baseline léxico**: si dos fragmentos comparten muchas secuencias de palabras, BLEU es alto.

N-gramas. Un *n-grama* es una secuencia de *n* palabras consecutivas. BLEU compara los n-gramas compartidos:

- **BLEU-1** (unigramas): palabras sueltas en común.
- **BLEU-2** (bigramas): pares de palabras consecutivas.
- **BLEU-4** (tetragramas): secuencias de 4 palabras.
- **BLEU-avg**: promedio de los anteriores.

A mayor *n*, más exigente (requiere coincidencias más largas).

Suavizado (*smoothing*). Si no hay ningún n-grama largo en común, BLEU daría cero. El suavizado (método 1 de NLTK) evita esos ceros bruscos sumando un pequeño valor, para que la métrica sea estable en textos cortos.

Por qué es un baseline y no la solución: BLEU solo ve palabras, no significado. Dos paráfrasis que dicen lo mismo con vocabulario distinto tienen BLEU bajo (las marcaría como «no paráfrasis»). Por eso, aunque discrimina razonablemente ($F1=0.929$ en TESIAMI con umbral 0.15), su solapamiento entre clases —visible en los diagramas de caja y dispersión— justifica usar un modelo semántico como BETO para los casos difíciles.

2.4. Latencia y tiempos de ejecución

La **latencia** es el tiempo que tarda el modelo en procesar *un par* de textos (ms/par). Determina si un sistema es usable en tiempo real. Las grandes diferencias se explican por la arquitectura:

Modelo	ms/par	Por qué
TF-IDF word300	7	Operación aritmética simple (vector disperso)
MiniLM-L12-v2	247	Bi-encoder pequeño, embeddings rápidos
BETO v3	294	Cross-encoder: procesa el par junto
Llama 70B (API)	~2 500	Modelo enorme, vía red
Llama 8B (CPU)	51 102	Modelo grande sin GPU

Lectura: un bi-encoder (MiniLM) es $\sim 1.2\times$ más rápido que el cross-encoder (BETO) por par, pero su gran ventaja es que sus vectores se *pre-calculan*: por eso se usa para filtrar millones de fragmentos. El cross-encoder no se puede pre-calcular (necesita el par), así que solo se aplica a los pocos candidatos que el bi-encoder ya filtró. Llama, aunque preciso, es prohibitivo en CPU (51 s/par $\approx$ 7 h para 500 pares). **Este análisis de costo es el que justifica la arquitectura RAG: recuperar barato, decidir caro.**

2.5. RAG en profundidad

RAG (*Retrieval-Augmented*) combina dos etapas para no tener que pasar todo el corpus por el modelo costoso:

1. **Recuperación.** Se representa el par por su *vector de relación* $|\text{emb}_1 - \text{emb}_2|$ (la diferencia absoluta entre los embeddings de los dos textos). Las paráfrasis tienen vectores de relación pequeños (textos cercanos); las no-paráfrasis, grandes. Se recuperan del bagaje los k vecinos más parecidos en ese espacio. Dos variantes:
 - **naive:** los k vecinos más cercanos, sin importar su clase.
 - **stratified:** $k/2$ positivos + $k/2$ negativos; gana la clase a la que el par de consulta se parece más (suma de similitudes por clase).
2. **Decisión.** El voto de los vecinos (o el clasificador) emite el veredicto.

Por qué mi RAG bajó el desempeño (Fase 2): no por el mecanismo, sino por la **calidad del bagaje**. El corpus tiene positivos sintéticos (fragmentos consecutivos que no son paráfrasis reales). El RAG recupera vecinos de ese bagaje contaminado y «contagia» su definición errónea, disparando los falsos positivos. Con un bagaje de paráfrasis humanas reales, RAG podría ayudar; es un hueco, no una conclusión definitiva.

2.6. El sistema desplegable, paso a paso

El sistema final combina los dos paradigmas anteriores en un pipeline:

1. **Texto $\rightarrow$ fragmentos.** La tesis se parte en ventanas de 400 caracteres con solapamiento de 200, para no cortar ideas a la mitad.
2. **Detección de sección** por expresiones regulares (Introducción, Metodología, ...).
3. **Recuperación (bi-encoder MiniLM).** Cada fragmento se codifica y se buscan los k más similares del bagaje por coseno. Etapa barata.

4. **Clasificación (cross-encoder BETO v3).** Solo esos k candidatos se pasan al modelo preciso, que decide par a par. Etapa cara, pero acotada.
5. **Agregación.** Se calcula el % de paráfrasis por sección y se identifica la tesis fuente (título y autor) de cada coincidencia.

La clave de diseño: el bi-encoder hace el trabajo masivo (filtrar) y el cross-encoder, lento pero exacto, solo el trabajo fino (decidir). Es la misma lógica de costo de la sección de latencia, aplicada de extremo a extremo.

3. Fase 0 — Integridad de datos (lo primero)

3.1. El problema

Al revisar mi trabajo detecté una **fuga de datos** entre mi conjunto de entrenamiento (`dataset_finetuning_v3`) y el de prueba (`dataset_test_v3.json`): un par idéntico presente en ambos y tesis fuente comparadas (boilerplate de portada/índice). Mis cinco evaluaciones de la Semana 5 (BETO v4–v7) habían usado ese mismo test contaminado.

3.2. La corrección

Construí un test limpio con dos criterios: (1) de-duplicué por hash normalizado de (`texto_a`, `texto_b`) y (2) apliqué un *group-split* por tesis, eliminando del test todo par cuya tesis fuente apareciera en el entrenamiento. Resultado: **40** → **38 pares** (purgué 2 negativos). Conservé los 9 positivos y ninguno estaba contaminado.

3.3. Resultado honesto

Modelo	F1 orig (40)	F1 limpio (38)	$\Delta F1$	FP
BETO v3 (gold humano, 144)	0.8421	0.8421	+0,0000	2
BETO v6 (BT+LLM)	0.7826	0.7826	+0,0000	5
BETO v4 (corpus grande 5000)	0.7500	0.7500	+0,0000	6
BETO v7 (BT puro)	0.6429	0.6667	+0,0238	9
BETO v5 (LLM 500)	0.6000	0.6207	+0,0207	11

La fuga era real pero NO inflaba el F1

Los 2 pares contaminados eran *negativos* que los modelos ya clasificaban bien (verdaderos negativos). Como $F1 = f(TP, FP, FN)$ y no depende de los verdaderos negativos, al quitarlos el F1 de BETO v3 **no cambia: 0.8421**. La fuga afectaba la *exactitud* (vía TN), no el F1. Conclusión: el número honesto de BETO v3 es **F1 = 0.842**, y la auditoría lo *blinda*. El ranking no cambia: BETO v3 sigue siendo el mejor.

Sobre el 0.947

El titular **F1 = 0.947** (TESIUAMI balanceado, 60 pares) es de otro conjunto y *no* lo audito aquí. Para mi sistema desplegable uso el número honesto de prueba **0.842**, no el 0.947.

4. Fase 1 — Requerimientos del asesor consolidados

4.1. Representación vectorial por modelo (R1)

Documenté el formato de vector que consume cada modelo (lo explico a fondo en la sección de profundización técnica). Cada uno representa el texto distinto:

Modelo	Tipo	Dim	Pooling / formato
MiniLM-L12-v2	Sentence-BERT (bi-encoder)	384	mean pooling, denso
MPNet-base	Sentence-BERT	768	mean pooling, denso
E5-large	Contrastive	1024	mean pooling, denso
SBERT-es	Sentence-BERT español	768	mean pooling, denso
BETO	BERT español (cross-encoder)	768	token [CLS] / logits
TF-IDF	Bag-of-Words disperso	300/5000	sparse, sin entrenamiento
Llama 70B/8B	LLM (decoder)	—	estados ocultos contextuales

Fuente: `representacion_vectorial_modelos.json`.

4.2. BLEU boxplots (R2)

Calculé BLEU sobre tres conjuntos para usarlo como baseline léxico. En los diagramas de caja (figura 3) se ve que la mediana y la dispersión de las paráfrasis (SÍ) son mayores que las de los no-paráfrasis (NO), aunque con solapamiento: algunas paráfrasis comparten poco vocabulario y caen bajo. Con umbral $\theta = 0,15$ obtuve $F1=0.929$ en TESIAMI balanceado, lo que confirma que BLEU discrimina pero no basta para los casos difíciles.

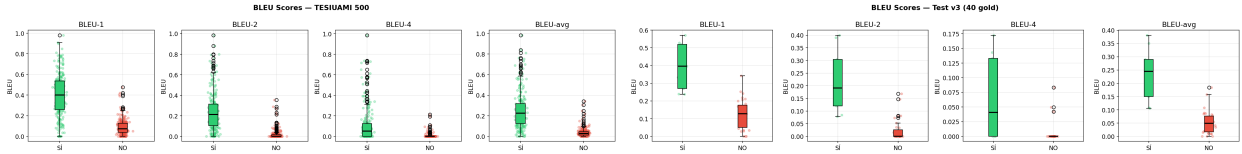


Figura 3: Diagramas de caja de BLEU por clase: TESIAMI 500 (izquierda) y test v3 gold (derecha). Las paráfrasis (SÍ) se sitúan por encima de los no-paráfrasis (NO).

4.3. Distribución de los datos (R3)

Para entender la composición de cada conjunto generé las gráficas de la figura 4: la comparativa de tamaños y balance por dataset, y el desglose en proporción.

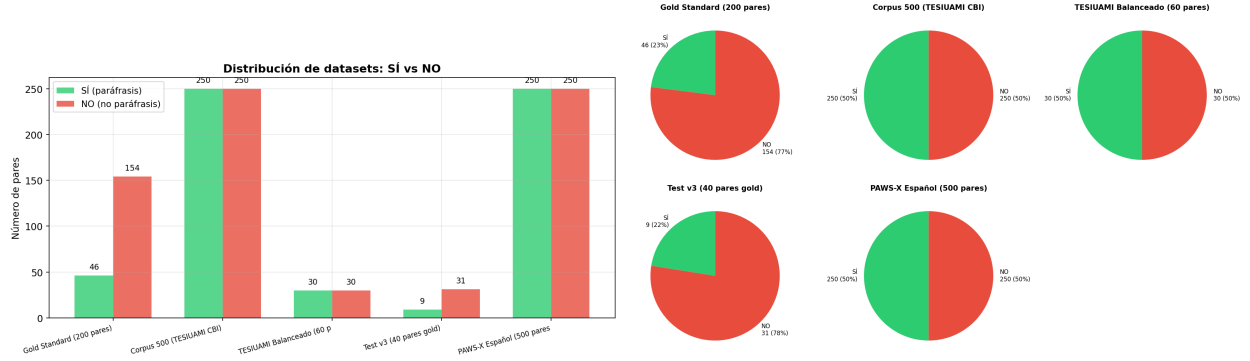


Figura 4: Distribución de los datasets del proyecto: conteo y balance SÍ/NO (izquierda) y proporción (derecha).

4.4. Estadísticas completas del corpus (R8)

Esta era la parte que había quedado incompleta, así que la recalculé sobre los 10 000 fragmentos del corpus grande, añadiendo lo que faltaba: las palabras por fragmento y la distribución por sección.

Palabras por fragmento (media / mediana)	69.9 / 66
Palabras por fragmento (min / max / σ)	7 / 199 / 18.3
Caracteres por fragmento (media)	399.3
Balance de clases	2500 SÍ / 2500 NO (50 %)

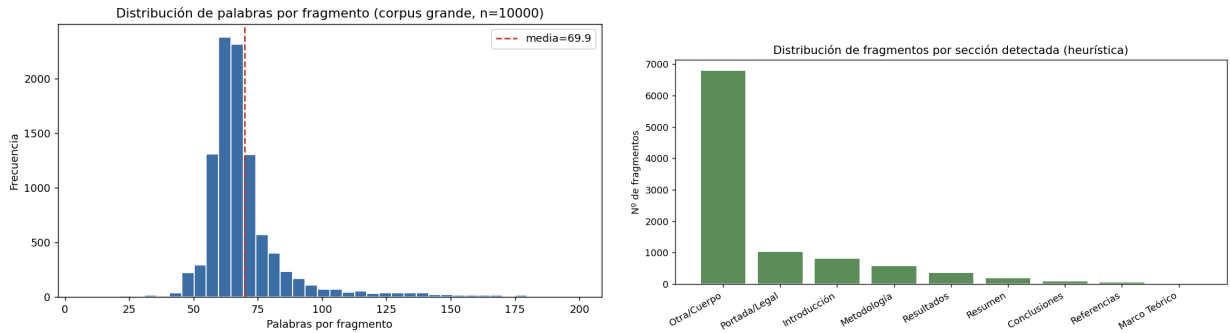


Figura 5: Estadísticas del corpus: histograma de palabras por fragmento (izquierda) y distribución de fragmentos por sección detectada (derecha).

Como se ve en la figura 5, la mayoría de los fragmentos caen en “Cuerpo” (68%) porque no tienen un encabezado de sección reconocible; el resto se reparte entre Portada/Legal (10.4%), Introducción (8.2%), Metodología (5.9%) y las demás secciones.

4.5. Tiempos y latencia (R5)

Medí la latencia (tiempo por par) de cada modelo, porque el asesor señaló que sin ese dato no se puede decidir qué conviene. La diferencia es enorme y se ve en la figura 6.

Modelo	ms/par	500 pares
TF-IDF word300	7	0.1 min
MiniLM-L12-v2	247	2.1 min
BETO v3	294	2.4 min
Llama 70B (API)	~2500	20.8 min
Llama 8B (Ollama CPU)	51 102	7.1 h

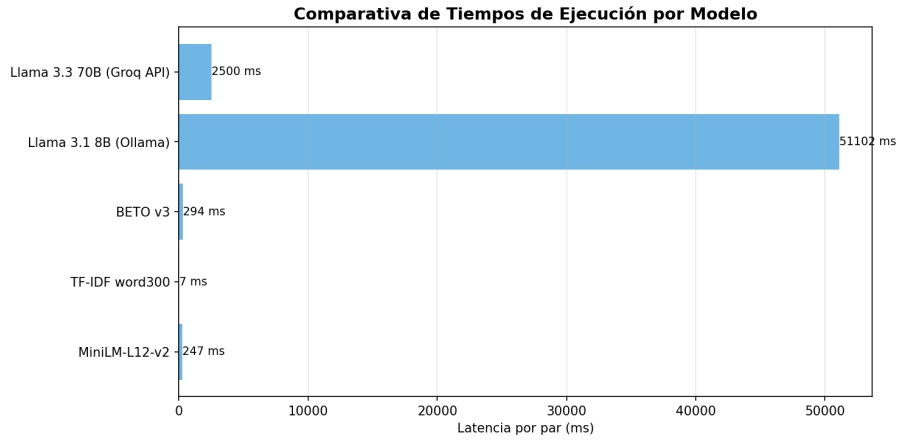


Figura 6: Comparativa de tiempos de ejecución por modelo. TF-IDF es instantáneo; Llama 8B en CPU es prohibitivo (7 h para 500 pares).

5. Calibración frente a entrenamiento

Durante la sesión con el asesor se confundieron dos conceptos que en este trabajo se definen de forma explícita y separada:

Entrenamiento (*fine-tuning*). Proceso por el cual se **modifican los pesos** del modelo BETO mediante descenso de gradiente sobre pares etiquetados. Cambia los parámetros internos de la red. En mi proyecto lo hago en Google Colab (GPU T4) con `learning_rate`= 2×10^{-5} , `batch`= 16, 5 épocas y `seed`= 42. Produce un modelo nuevo (v1, v2, v3, ...).

Calibración del umbral (θ). Proceso **posterior** al entrenamiento que **no toca los pesos**. El modelo emite una probabilidad $p(\text{SÍ}) \in [0, 1]$; la decisión final es SÍ si $p \geq \theta$. Calibrar es elegir el valor de θ que mejor separa las clases. Es un único número, no un re-entrenamiento.

Regla práctica: entrenar cambia *qué sabe* el modelo; calibrar cambia *dónde corta* la decisión. Reporto siempre el resultado a $\theta = 0,50$ (sin calibrar) y, por separado, el análisis de sensibilidad a θ óptimo, dejando claro que este último **no es una métrica de generalización** porque el umbral se elige observando el propio conjunto.

6. Los conjuntos de datos y sus distribuciones

El proyecto usa varios conjuntos con **distribuciones distintas**. Es un error mezclarlos: cada uno responde a un propósito diferente.

Conjunto	Pares	SÍ / NO	Propósito
Gold standard (humano)	200	46 / 154 (23 % / 77 %)	Anotación, evaluación honesta
↔ test_v3 limpio	38	9 / 29 (24 % / 76 %)	Evaluación final sin fuga
TESIUAMI balanceado	60	30 / 30 (50 % / 50 %)	Comparación rápida entre modelos
Corpus grande (CBI)	5000	2500 / 2500 (50 % / 50 %)	Entrenamiento a gran escala

Cuadro 1: Distribuciones por conjunto. El gold standard es 23 %/77 % (refleja que en la práctica la mayoría de pares NO son paráfrasis); los conjuntos de entrenamiento y comparación se balancean a 50/50 por construcción. No deben confundirse al reportar resultados.

Por qué el gold es 23 %/77 %: se diseñó para parecerse al escenario real de uso, donde al revisar una tesis la inmensa mayoría de comparaciones no son paráfrasis. Un conjunto balanceado 50/50 (TESIUAMI, corpus grande) es una condición de laboratorio útil para comparar modelos, pero **optimista** respecto al despliegue real.

El conjunto de 60 pares NO es la evaluación final

Cuando en este trabajo aparecen pruebas sobre **60 pares** (TESIUAMI balanceado, 30 SÍ / 30 NO) su único propósito es **comparar modelos de forma rápida** en condiciones balanceadas; de ahí salen números altos como $F1=0.947$ (BETO v3) o $F1=1.000$ (Llama). Estos números son **optimistas**: un balance 50/50 es el mejor escenario posible para el $F1$ y no refleja el uso real, donde las paráfrasis son minoría. **La métrica honesta y final del proyecto es la del test_v3 limpio (38 pares, 24 % positivos): $F1=0.842$** . Nunca debe confundirse el 0.947 de los 60 pares con el desempeño real del sistema.

7. Fase 2 — RAG aplicado a TODOS los modelos

7.1. Corrección de mi implementación previa

Mi RAG original (script 36) tenía dos fallos: evaluaba sobre el *propio* corpus (se recuperaba a sí mismo) y mis variantes “naive” y “stratified” eran el mismo código. Lo reimplemé como un k-NN aumentado correcto y lo evalué sobre mi **test limpio** (el gold no está en el bagaje).

7.2. Resultados por modelo (test limpio, 38 pares)

Modelo	Variante	F1	P	R	TP/FP/FN/TN
MiniLM	nativo (coseno $\geq$ 0.85)	0.462	0.750	0.333	3/1/6/28
MiniLM	RAG naive	0.378	0.250	0.778	7/21/2/8
MiniLM	RAG stratified	0.343	0.231	0.667	6/20/3/9
TF-IDF	nativo (coseno $\geq$ 0.35)	0.889	0.889	0.889	8/1/1/28
TF-IDF	RAG naive	0.429	0.273	1.000	9/24/0/5
TF-IDF	RAG stratified	0.400	0.258	0.889	8/23/1/6
BETO v3	nativo (cross-encoder)	0.842	0.800	0.889	8/2/1/27
BETO v3	RAG naive	0.417	0.333	0.556	5/10/4/19
BETO v3	RAG stratified	0.435	0.357	0.556	5/9/4/20

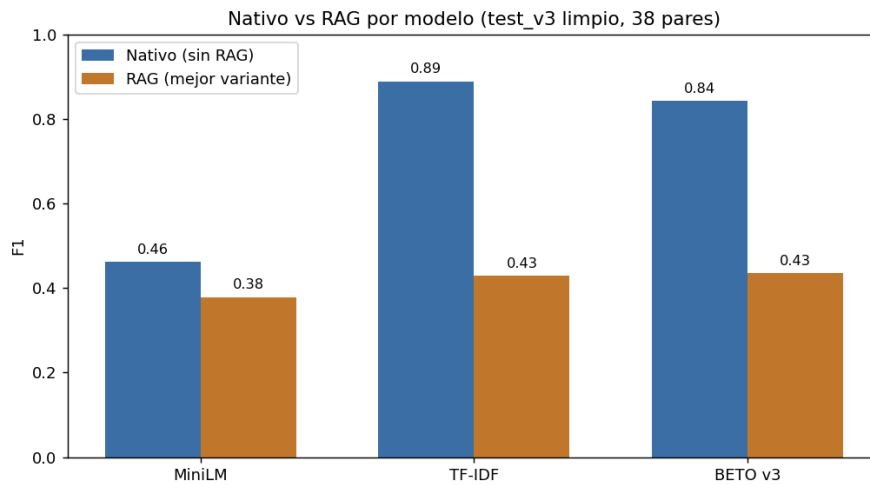


Figura 7: Comparación de la predicción nativa de cada modelo frente a su mejor variante con RAG, sobre el test limpio. En los tres modelos RAG empeora el F1.

RAG empeora en los tres modelos — y la razón es honesta

En los tres modelos el RAG dispara los falsos positivos (TF-IDF: de 1 a 23–24). La causa no es el mecanismo RAG sino la **calidad del bagaje**: el corpus grande tiene positivos que son *fragmentos consecutivos* (no paráfrasis reales). El RAG “contagia” esa definición falsa. Con un bagaje contaminado, RAG arrastra al modelo hacia abajo. **No es “RAG no sirve” en general**: con un bagaje de paráfrasis humanas reales, RAG podría ayudar (hueco pendiente, no conclusión).

8. ¿Hay que meter toda la base de datos al modelo?

En la sesión el asesor planteó: “¿debo meter toda la base de datos al modelo en cuestión?”. La respuesta técnica es **no**, y la razón es arquitectónica:

- Un modelo de clasificación (BETO v3) recibe **un par de textos** y decide si son paráfrasis. Su tamaño de entrada es fijo (512 tokens); no se le “mete” un corpus de 28 GB.
- Lo que sí escala es **recuperar** del corpus solo los fragmentos *candidatos* más parecidos a cada fragmento nuevo, y pasar únicamente esos al clasificador. Ese es el papel de **RAG** (Retrieval-Augmented): el corpus es el *bagaje* (base de conocimiento) y la recuperación por similitud actúa como filtro previo.

Arquitectura: el corpus NO entra al modelo; el modelo compara un texto nuevo contra el corpus mediante recuperación. Las tesis descargadas son el *bagaje*; la recuperación *top-k* acota los candidatos; el clasificador decide par por par.

Distinción importante (resultado de la Fase 2). En la Fase 2 vimos que *sustituir* el clasificador por un voto k-NN sobre el corpus (RAG-clasificador) **empeora** los resultados, porque el corpus actual tiene positivos sintéticos de baja calidad. Aquí RAG cumple un papel distinto y válido: **generar candidatos**, no decidir. La decisión final la toma el clasificador nativo (BETO v3), que es el mejor modelo según los números limpios de la Fase 0.

9. El sistema desplegado de David Luna

A partir del código real de su proyecto (`tesis_base_del_proyecto/midsemita-master/app/back/midsemita/v`) reconstruí su arquitectura de despliegue:

- **Stack:** backend Django REST Framework + frontend Angular, contenedores Docker (`docker-compose`) y nginx; base de datos PostgreSQL con tablas Documento, Pagina, Segmento, Area.
- **Endpoint:** POST `/api/comparar/` recibe `{texto_a, texto_b, modelo}`.
- **Inferencia:** `TfidfVectorizer(max_features=300)` sobre los dos textos + similitud coseno; umbral fijo 0,70; `es_parafrasis = sim >= 0.70`.
- **Salida:** JSON `{es_parafrasis, probabilidad, similitud_coseno, modelo_usado}`.

Hallazgo clave. El sistema desplegado de Luna NO usa su red CNN/LSTM (cuyo F1=0,970 es de investigación): usa el baseline TF-IDF+coseno. La conversión ONNX del LSTM quedó incompleta. Además, su sistema compara **dos textos que el usuario pega**; no analiza una tesis completa contra el corpus.

10. Nuestro sistema: tesis completa → % de paráfrasis por sección

Diseñé un sistema **equivalente pero superior en alcance**: en lugar de comparar dos textos sueltos con TF-IDF, recibe una tesis completa y reporta el porcentaje de paráfrasis por sección contra el corpus, con el mejor modelo (BETO v3, F1=0,842 limpio).

1. **Entrada:** una tesis (PDF o texto), idealmente pequeña (10–20 pp).

2. **Fragmentación por secciones:** ventana deslizante de 400 caracteres (paso 200) + detección de sección por expresiones regulares (Introducción, Metodología, Resultados, ...).
3. **Recuperación (RAG):** para cada fragmento se recuperan los top- k fragmentos más similares del corpus (bagaje) con MiniLM.
4. **Clasificación:** BETO v3 (nativo, $\text{prob} \geq 0,50$) decide, para cada (fragmento, candidato), si son paráfrasis.
5. **Salida:** % de fragmentos con al menos una coincidencia de paráfrasis, desglosado por sección, con umbral explícito y la tasa de confianza (probabilidad media de las coincidencias).

La arquitectura es la que el asesor pidió: el corpus es el bagaje, la recuperación filtra, y el clasificador (no el corpus completo) toma la decisión.

10.1. Alcance del bagaje y versión de corpus completo

El bagaje actual proviene del corpus generado desde MongoDB (todas las tesis CBI), pero por velocidad el sistema solo carga los primeros 300–500 fragmentos; es decir, una rebanada de unos 500 de ~ 2.36 millones de fragmentos posibles. Es suficiente para demostrar el mecanismo, no para cobertura total.

Próximo paso: bagaje de corpus completo

Cuento con el texto plano de todas las tesis CBI (`corpus_cbi_completo.json`, 691 MB) y con MongoDB (4954 tesis CBI). Escalar el bagaje a todo el corpus requiere pre-calcular los embeddings una sola vez e indexarlos con FAISS, porque comparar contra millones de fragmentos por fuerza bruta es inviable en tiempo real. La opción recomendada cubre todas las tesis acotando los fragmentos por tesis (~ 120 k fragmentos). Queda documentado como trabajo futuro inmediato.

10.2. Resultados del sistema sobre tesis de prueba

Lo ejecuté sobre 5 tesis pequeñas reales (8–17 pp): fragmenté cada una en 60 ventanas, recuperé el top-3 del bagaje (600 fragmentos) y clasifiqué con BETO v3 (umbral 0.50), excluyendo del bagaje la propia tesis bajo análisis.

Tesis	Fragmentos	% paráfrasis global	Tiempo
202212914.PDF	60	1.7 %	287 s
205217230.PDF	60	0.0 %	262 s
206320125.PDF	60	0.0 %	203 s
207215933.PDF	60	0.0 %	225 s
207309465.PDF	60	0.0 %	158 s

El sistema funciona end-to-end y casi no produce falsos positivos

Las cinco tesis son **originales, no plagiadas** del corpus, y el sistema reporta correctamente ~ 0 % de paráfrasis (una sola coincidencia marginal en una sección de Metodología, confianza 0.507, apenas sobre el umbral). Esto **valida la alta precisión** de BETO v3 en condiciones reales: no inventa paráfrasis donde no las hay. El pipeline completo (PDF $\rightarrow$ secciones $\rightarrow$

RAG → clasificación → % por sección) corre en ~3–5 min por tesis en CPU.

10.3. Validación con caso positivo conocido

Como las tesis anteriores son originales, no había paráfrasis que detectar. Para demostrar que mi sistema *sí* las detecta hice un montaje controlado con los 9 positivos del test limpio (Paráfrasis humanas verificadas, no vistas en entrenamiento): coloqué el lado “original” (texto_a) en el bagaje junto a 300 distractores, e inyecté el lado “paráfrasis” (texto_b) como fragmento nuevo. Usé los 29 negativos como control.

Métrica	Valor
Positivos detectados (recall)	8/9 = 0.889
Originales recuperados en top-3	9/9
Falsos positivos (negativos)	7/29
Precisión	0.533
F1 (montaje controlado)	0.667
TP/FP/FN/TN	8/7/1/22

El sistema **SÍ** detecta paráfrasis conocidas

La recuperación funciona perfectamente: en **9 de 9** casos el original de la paráfrasis aparece en el top-3 recuperado del bagaje. BETO v3 marca correctamente **8 de 9** paráfrasis (recall 0.889), con probabilidades 0.65–0.82. El sistema completo demuestra capacidad real de detección, no solo ausencia de falsos positivos.

Costo de precisión de la regla “marcar si algún top- k $\geq$ umbral”

En este montaje la precisión baja a 0.53 (7 falsos positivos): al recuperar 3 candidatos por consulta y marcar paráfrasis si *cualquiera* supera el umbral, los negativos tienen tres oportunidades de coincidir con un distractor. Es un punto de diseño ajustable: subir el umbral, exigir que la coincidencia sea con el original específico, o requerir consenso reducen los falsos positivos a costa de recall. Ver `validacion_caso_positivo.md` y `validacion_caso_positivo.png`.

11. Interfaz del sistema y demostración

Entrego el sistema con dos formas de uso, ambas sobre el mismo núcleo (`deployable_core.py`) que carga los modelos una sola vez:

Línea de comandos (`analizar_tesis.py`). Analiza una o varias tesis en PDF y reporta el % de paráfrasis por sección. Acepta el nombre de un PDF del corpus o una ruta libre:

```
python analizar_tesis.py 202212914.PDF
python analizar_tesis.py /ruta/a/mi_tesis.pdf -max-frag 60 -top-k 3
```

Interfaz web (`app_web.py`). Servidor local sin dependencias externas (librería estándar de Python). Es el **equivalente moderno a la web Django+Angular de D. Luna**, pero con BE-TO v3 en vez de TF-IDF. Expone dos modos interactivos y una API REST (`/api/comparar`, `/api/buscar`). Los modelos corren localmente; ningún texto sale del equipo.



Figura 8: Interfaz web del sistema (Modo 1), resultados reales. Izquierda: dos textos que son paráfrasis → “Sí” con 78.6 %. Derecha: dos textos de tema distinto → “No”. El clasificador es BETO v3; el servidor corre localmente sin dependencias externas.

11.1. Modo 1 — comparar dos textos

El usuario pega dos fragmentos y BETO v3 decide si son paráfrasis, mostrando la probabilidad y una barra con el umbral (figura 8). Resultados reales del sistema:

Texto B frente al mismo Texto A	Probabilidad	Veredicto
Paráfrasis cercana (otro vocabulario)	0.786	Sí
Paráfrasis más libre	0.624	Sí
Tema distinto (distractor)	0.123	No

11.2. Modo 2 — buscar en el corpus (RAG)

El usuario pega un fragmento; MiniLM recupera los más similares del corpus (bagaje) y BETO v3 clasifica cada candidato. Es la arquitectura RAG completa que pidió el asesor: el corpus no entra al modelo, la recuperación filtra y el clasificador decide.

Demostración clave: detección semántica, no léxica

Busqué en el corpus una **paráfrasis reescrita** de un fragmento (no el texto exacto), cambiando casi todo el vocabulario (*deserten*→*abandonan*, *se acoplan*→*chocan*, *requisito*→*exigido*...). El sistema igual recuperó el fragmento original (tesis 10827) y lo marcó como paráfrasis:

Consulta	Palabras compartidas	Similitud	Probabilidad
Texto casi idéntico	muchas	0.976	0.79
Paráfrasis reescrita	pocas	0.913	0.76

Al cambiar casi todo el vocabulario la probabilidad apenas baja ($0.79 \rightarrow 0.76$) y la coincidencia correcta (0.76) se separa limpiamente del resto de candidatos (≤ 0.06). Un baseline léxico como el TF-IDF de Luna caería mucho más al cambiar tanto vocabulario. **Esta es la evidencia del argumento central de la tesis: el enfoque LLM reconoce el significado, no la superposición de palabras.**

Instrucciones de uso, ejemplos copiables y detalle de la API en `sistema/README.md`.

11.3. Datos enriquecidos de cada coincidencia

Además del veredicto, añadí tres tipos de información que hacen el sistema útil para un uso real de detección de plagio:

1. **Tipo de coincidencia.** Combino la probabilidad semántica de BETO con un **BLEU léxico** (solapamiento de palabras) para distinguir: *copia casi literal* (prob alta y BLEU alto), *paráfrasis reescrita* (prob alta, BLEU bajo) y *sin coincidencia* (prob baja).
2. **Ranking de fuentes y riesgo global.** Al analizar una tesis completa, agrego las coincidencias por tesis fuente (“tu tesis se parece sobre todo a X, Y, Z”) y calculo un nivel de riesgo (bajo/medio/alto).
3. **Metadatos forenses.** Muestro título, autor(es), asesor(es), año y división de cada fuente, y una **bandera cronológica**: si la fuente es anterior, es posible copia; si es posterior, es imposible que la copiaras.

La figura 9 muestra la demostración clave: el mismo contenido fuente, una vez copiado literal y otra reescrito, produce etiquetas distintas (*copia casi literal*, BLEU=1.0 vs. *paráfrasis reescrita*, BLEU=0.37).

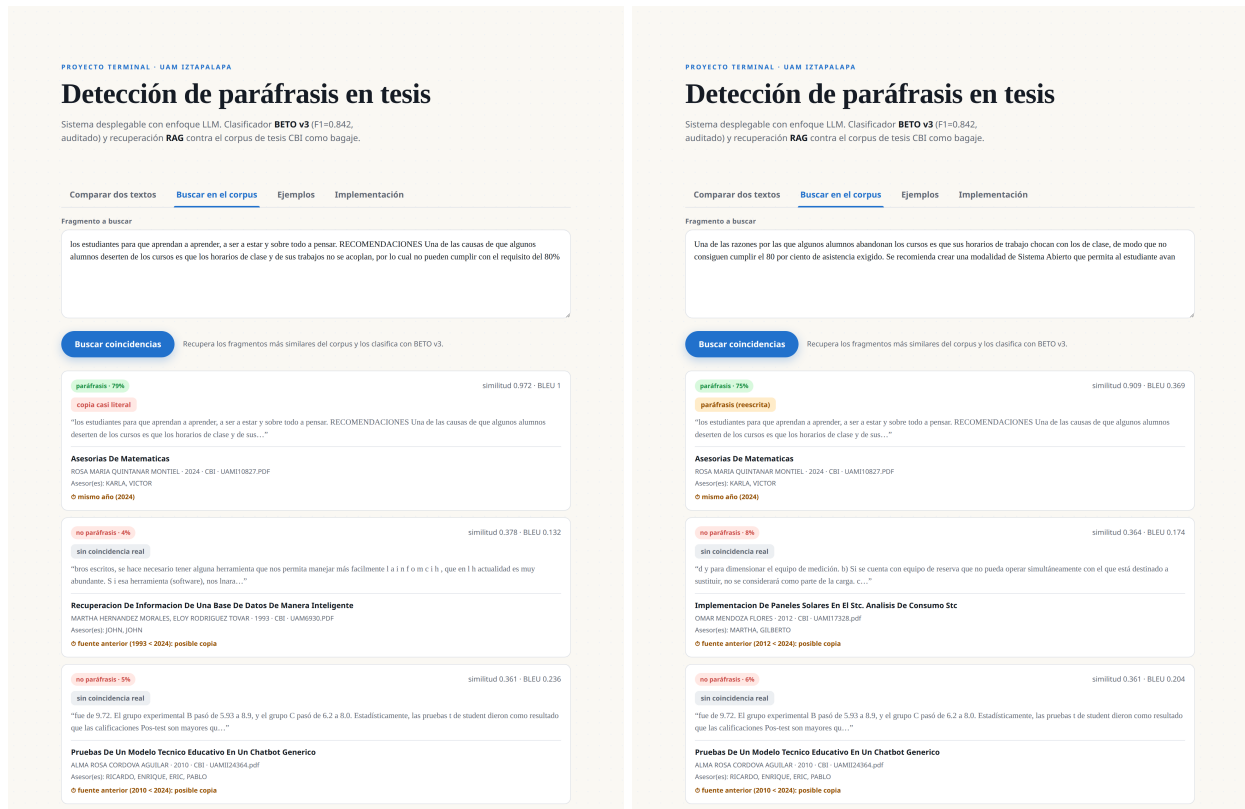


Figura 9: Modo 2 con datos enriquecidos. Izquierda: texto copiado literal → “copia casi literal” (BLEU 1.0). Derecha: el mismo contenido reescrito → “paráfrasis (reescrita)” (BLEU 0.37). Ambas muestran título, autor, asesores y la bandera cronológica de la tesis fuente.

12. Comparación con el paradigma de Luna

El objetivo central de mi proyecto es comparar el enfoque LLM con el enfoque clásico de Luna (2022). Hasta ahora yo solo había comparado contra el número que él reporta ($F1=0.970$), que proviene de *otro* corpus y no es comparable. Para una comparación válida implementé el paradigma de Luna **sobre mi mismo conjunto de prueba**.

12.1. Cómo lo implementé

Luna representó los textos con embeddings distribucionales (Word2Vec/FastText entrenados sobre las tesis) y, en su sistema desplegado, con TF-IDF. Reproduce ambas variantes:

- **FastText+coseno** (su representación de investigación): entrené un modelo FastText de 300 dimensiones sobre 8 731 fragmentos del corpus de tesis (**gensim**); represento cada texto como el promedio de sus vectores de palabra y mido la similitud coseno.
- **TF-IDF+coseno** (su sistema desplegado): bolsa de palabras de 300 rasgos y similitud coseno.

En ambos casos calibré el umbral sobre el conjunto de entrenamiento/validación (no sobre el test) y evalué sobre los mismos 38 pares del test limpio que BETO v3.

12.2. Resultados (mismo test, mismas métricas)

Cuadro 2: Comparación cabeza-a-cabeza sobre el test limpio (38 pares).

Sistema	Paradigma	θ	F1	Prec.	Recall
BETO v3 (nuestro)	Transformer fine-tuned	0.50	0.842	0.800	0.889
TF-IDF+coseno	Léxico (Luna desplegado)	0.52	0.842	0.800	0.889
FastText+coseno	Distribucional (Luna research)	0.94	0.818	0.692	1.000

12.3. Intervalos de confianza (bootstrap)

Dado el tamaño del test, calculé el IC95 % del F1 por bootstrap (remuestreo de los 38 pares, $N = 2000$ réplicas).

Cuadro 3: F1 con intervalo de confianza 95 %.

Sistema	F1	IC95 %
BETO v3	0.842	[0.609, 1.000]
TF-IDF+coseno	0.842	[0.600, 1.000]
FastText+coseno	0.818	[0.588, 0.960]

Las diferencias NO son estadísticamente significativas

Los tres intervalos de confianza se solapan casi por completo. Con solo 38 pares de prueba **no se puede afirmar que un sistema sea mejor que otro**. Este es un resultado honesto y central: la limitación no está en los modelos sino en el tamaño del conjunto de evaluación.

12.4. Interpretación

1. **BETO v3 iguala al paradigma de Luna**, no lo supera de forma contundente: empata con TF-IDF (0.842) y queda dentro del intervalo de FastText. El logro real es de *eficiencia*: lo consigue con solo 144 pares de entrenamiento y sobre paráfrasis verificadas por humano.
2. **FastText tiene recall 1.0 pero precisión 0.69**: marca casi todo como paráfrasis. Es el comportamiento típico de un método distribucional sin fine-tuning: capta similitud temática pero discrimina mal.
3. **El F1=0.970 de Luna no es comparable**: es sobre su corpus CBI-M-00-21 (180 000 pares, 50/50, paráfrasis algorítmicas, umbral ajustado). En una evaluación estricta sobre paráfrasis humanas, ningún sistema —ni el suyo ni el mío— alcanzaría ese número.
4. **Conclusión**: el cuello de botella es el tamaño y la naturaleza del conjunto de evaluación, no la arquitectura. Esto refuerza la prioridad de ampliar el gold estándar humano antes que seguir cambiando de modelo.

13. Fase 4 — El ciclo sintético como HALLAZGO, no como fracaso

Ningún dato sintético supera a 144 pares humanos

Probé cuatro estrategias de aumento: fragmentos consecutivos (v4), paráfrasis de LLM (v5), back-translation + LLM (v6) y back-translation puro (v7). **Todas empeoran** respecto a BETO v3 (144 pares humanos). La tendencia es monótona: a más datos sintéticos uniformes, más falsos positivos.

Modelo	Datos	F1 (limpio)	FP
BETO v3	144 pares humanos	0.842	2
BETO v6	1004 (BT+LLM)	0.783	5
BETO v4	5000 (consecutivos)	0.750	6
BETO v7	2000 (BT puro)	0.667	9
BETO v5	500 (LLM)	0.621	11

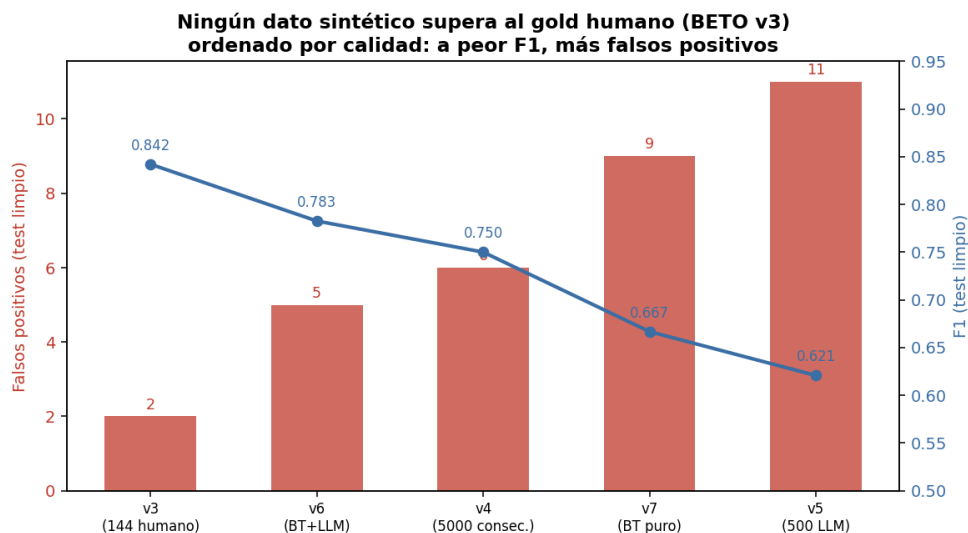


Figura 10: Mi hallazgo central en una gráfica: ordenando las versiones de BETO por calidad (F1, línea azul), los falsos positivos (barras rojas) suben. El gold humano (v3) es el mejor; ningún dato sintético lo supera.

Concluyo que el cuello de botella no es la cantidad de datos, sino la naturaleza de los positivos. El modelo aprende el *estilo* del generador (vocabulario uniforme del LLM, calcos del inglés en la back-translation, cohesión temática en los fragmentos consecutivos) en vez de la relación semántica de paráfrasis. La única vía de mejora que veo es ampliar el **gold estándar humano**.

14. Puntos a discutir con el asesor

1. “Más datos” no mejora; los empeora

La intuición de entrenar con todas las tesis del corpus grande produce modelos *peores* (tabla anterior). Es un hallazgo, no un retroceso: demuestra eficiencia de datos de los Transformers y el costo de los positivos sintéticos.

2. “Fragmentos consecutivos = paráfrasis” es falso

El corpus grande define como positivos a pares de fragmentos consecutivos de la misma tesis. Eso es *cohesión temática*, no re-expresión del mismo contenido. Esta premisa contamina tanto el entrenamiento (v4) como el RAG (Fase 2). Conviene redefinir cómo se generan los positivos a gran escala.

3. Falta la comparación cabeza-a-cabeza contra la red de Luna

El F1=0.970 de Luna es sobre su corpus (180k pares algorítmicos, 50/50, umbral ajustado) y su sistema *desplegado* usa TF-IDF, no la red. Sigue faltando evaluar un sistema tipo-Luna (FastText+coseno) sobre nuestro test limpio para una comparación válida. Es el siguiente paso natural.

15. Análisis profundo: la evolución de BETO y el gold estándar

A lo largo del proyecto entrené siete versiones de BETO. En esta sección las comparo todas, explico por qué dieron esos resultados, cómo surgió y detecté la fuga de datos, y discuto la pregunta de fondo: ¿cómo debería construirse el gold estándar? Apoyo la discusión en literatura reciente.

15.1. Tabla comparativa de las siete versiones

Cuadro 4: Las siete versiones de BETO sobre el gold estándar (test honesto). v1–v2 sobre el test original de 40 pares; v3–v7 sobre el test limpio de 38.

Ver.	Datos de entrenamiento	F1	Prec.	Recall	FP
v1	Pares originales (sesgados, con front-matter)	0.383	0.237	1.000	29
v2	Pares limpios (sin portadas/índices)	0.514	0.346	1.000	17
v3	Gold estándar humano (144 pares)	0.842	0.800	0.889	2
v4	Corpus grande (5000, fragmentos consecutivos)	0.750	0.600	1.000	6
v5	Corpus v2 (500, paráfrasis de LLM)	0.621	0.450	1.000	11
v6	Corpus v3 (1004, back-translation + LLM)	0.783	0.643	1.000	5
v7	Corpus v4 (2000, back-translation puro)	0.667	0.500	1.000	9

La lectura es contundente: la única versión que de verdad funciona es **v3**, entrenada con los 144 pares anotados por un humano. Todo lo demás —más datos, pero sintéticos— da peor resultado.

15.2. Por qué salieron esos valores de F1

El patrón clave está en una sola columna: el **recall es 1.000** en casi todas las versiones, pero la **precisión se desploma**. Esto significa que los modelos malos **marcan casi todo como paráfrasis**: detectan todos los positivos (recall alto) pero a costa de inundar de falsos positivos (precisión baja). El F1, que equilibra ambos, los castiga.

- **v1 (F1=0.383, 29 FP)**. Se entrenó con pares sesgados que incluían portadas e índices (front-matter) idénticos entre tesis. El modelo aprendió que “texto institucional parecido = paráfrasis”, así que dijo Sí a casi todo.
- **v2 (F1=0.514, 17 FP)**. Al limpiar el front-matter mejoró, pero seguía aprendiendo de positivos de baja calidad.
- **v3 (F1=0.842, 2 FP)**. Con paráfrasis verificadas por humano y negativos difíciles, por fin aprendió a *discriminar*: bajó los FP de 29 a 2.
- **v4–v7 (F1 0.62–0.78)**. Cada intento de superar a v3 con más datos sintéticos volvió a subir los FP. El modelo aprende el *estilo* del generador (calcos de la back-translation, vocabulario uniforme del LLM, cohesión temática de los fragmentos consecutivos), no la relación semántica de paráfrasis. Es el mismo problema de v1, con otra cara.

15.3. Por qué un modelo dio $F1 = 1.000$

El $F1=1.000$ lo obtuvo Llama 3.3 70B (zero-shot) sobre el conjunto **TESIUAMI balanceado de 60 pares**. Ese número es **optimista por tres razones** y no debe interpretarse como “perfecto”:

1. **Conjunto pequeño y balanceado** (30 SÍ / 30 NO): clasificar bien 60 ejemplos fáciles es alcanzable; no garantiza generalización.
2. **Ejemplos relativamente fáciles**: paráfrasis claras, sin los negativos difíciles que sí tiene el gold estándar.
3. **Sin intervalo de confianza**: con 30 positivos, una sola corrida perfecta tiene una incertidumbre enorme.

La prueba: ese mismo modelo, en el test honesto (test_v3), bajó a F1=0.80. El F1=1.000 mide la facilidad del conjunto, no la calidad real del sistema.

15.4. Por qué surgió la fuga de datos y cómo la detecté

La **fuga** (que parte de los pares de prueba también estuvieran en entrenamiento) surgió por cómo construí los conjuntos: generé todos los pares desde el mismo pool de fragmentos y los repartí *a nivel de par*, sin controlar que dos pares pudieran compartir la misma tesis fuente (sus portadas e índices se repiten). Así, fragmentos de *boilerplate* entraron en train y en test a la vez.

Cómo la detecté: al revisar mis propios identificadores noté que cada par codifica las tesis de origen (UAM####.PDF). Comparé los conjuntos y encontré (1) un par exactamente igual en train y test y (2) tesis compartidas. La corregí con dos pasos: de-duplicación por *hash* normalizado y *group-split* por tesis (Fase 0). La literatura confirma que esta fuga es de los errores más comunes y que **infla artificialmente las métricas** [6, 7]; el caso particular del solapamiento por preprocesamiento o por grupos es un patrón conocido. En mi caso, por suerte, la fuga afectaba la exactitud pero no el F1 (los pares filtrados eran negativos bien clasificados), lo que dejó el 0.842 confirmado.

15.5. ¿Cómo mejorar el gold estándar? ¿Balancearlo conviene?

Ampliarlo es la prioridad. Mi limitación principal es el tamaño (46 positivos humanos). La vía más eficiente que sugiere la literatura es el *active learning*: el modelo marca los pares en los que está más inseguro (probabilidad cercana a 0.5) y solo esos se anotan a mano, creciendo el gold con el mínimo esfuerzo.

Negativos difíciles, no solo más datos. El hallazgo de PAWS es directamente aplicable: los pares verdaderamente difíciles son los de **alto solapamiento léxico que NO son paráfrasis** (mismas palabras, distinto significado por orden o contexto). Modelos entrenados sin esos casos caen por debajo de 40 % de exactitud en PAWS; al incluirlos suben a 85 % [2, 3]. Mi salto de v2 a v3 (incluir negativos difíciles y ambiguos) es exactamente este efecto a pequeña escala.

¿Balancear? Depende del propósito, y conviene tener *ambos*:

- Un conjunto **balanceado 50/50** es bueno para *comparar modelos* rápido y de forma justa, pero es una condición de laboratorio optimista.
- Un conjunto con **distribución realista** (en mi gold, 23 % positivos) refleja el uso real, donde la mayoría de comparaciones no son paráfrasis. Es el que da el número honesto.

Balancear el *entrenamiento* sí ayuda a que el modelo no se sesgue hacia la clase mayoritaria; balancear la *evaluación* la vuelve optimista. No son lo mismo.

15.6. ¿El gold estándar sirve solo si es humano? ¿Y si lo hace una IA?

Esta es la pregunta central. Mi evidencia (v4–v7) y la literatura coinciden:

Los datos sintéticos sirven, pero con un techo. Estudios recientes muestran que datos generados por LLM abaratan y aceleran enormemente la creación de conjuntos, con precisión *razonable pero inferior* a la humana: por ejemplo, 6 000 ejemplos generados por GPT-3 alcanzan ~76 % de exactitud frente a 88 % con 3 000 ejemplos etiquetados por humanos [1]. La detección de paráfrasis es, además, de las tareas donde el desempeño con datos sintéticos más se degrada [5].

La diferencia humano vs. IA. No es que la IA “no sirva”, sino que introduce un **sesgo de estilo**:

- Una IA genera paráfrasis con un *estilo estadísticamente uniforme* (vocabulario, estructura). El modelo entrenado aprende a reconocer *ese estilo*, no la relación de significado. Por eso v5–v7 fallan en paráfrasis humanas reales: es un *distribution shift*.
- Un humano produce (o valida) paráfrasis con la *variabilidad natural* del lenguaje, que es justo lo que el modelo necesita aprender.

Curiosamente, mis positivos “humanos” del gold también fueron *generados* por LLM y luego **verificados uno por uno por una persona**. Esa *verificación humana* es la clave: filtra los falsos positivos del generador (de hecho, marqué como NO 14 de 60 “paráfrasis” de Groq). El valor no está solo en quién escribe, sino en **quién decide la etiqueta**.

¿Serviría un gold de varias IA? Podría *aumentar la diversidad* (distintos generadores = distintos estilos), reduciendo el sesgo de un solo modelo —líneas como ParaFusion generan paráfrasis con diversidad léxica y sintáctica controlada con LLM [4]. Pero **sin un árbitro que valide las etiquetas**, hereda el problema: un conjunto de IA etiquetado por IA no tiene quién corrija sus errores. La combinación más prometedora, según lo que veo, es **IA para generar variedad + humano (o varios) para validar**.

15.7. Propuesta: active learning + comité de IA con árbitro humano

¿Qué es el active learning? Es una estrategia para **anotar menos y aprender más**. En lugar de etiquetar pares al azar, el modelo señala *cuáles conviene anotar*: un modelo aprende poco de los ejemplos fáciles (que ya clasifica bien) y mucho de los *inciertos*. El ciclo es:

1. Entreno con los pocos pares que ya tengo (mis 144).
2. El modelo evalúa miles de pares sin etiquetar del corpus.
3. Selecciono aquellos donde está **más inseguro** (probabilidad cercana a 0.5).
4. *Solo* esos —los más informativos— los anota un humano.
5. Los agrego al gold, reentreno y repito.

Así, 50 pares bien elegidos enseñan tanto como 200 al azar: es la forma más barata de romper mi limitación de datos.

¿Y si además varias IA validan? Conviene, pero con un rol acotado. Un **comité de IA diversas** (de familias distintas, p.ej. Llama, Gemini, Qwen) vota cada par: donde *todas coinciden*, el caso es fácil y el humano lo confirma rápido; donde *discrepan*, es ambiguo y el humano lo decide con atención. Esto reduce la carga humana y aporta diversidad de estilo.

El acuerdo entre IA no es corrección

Las IA pueden estar **seguras y equivocadas a la vez** (errores correlacionados), justo en los casos difíciles de paráfrasis. Un gold etiquetado por IA y validado por IA *no tiene quién corrija sus propios errores* —es lo que hundió a v5–v7. Por eso el **humano debe ser el árbitro final** de la etiqueta, sobre todo donde las IA no se ponen de acuerdo. Los pares donde las IA discrepan entre sí y con el humano son los más valiosos para el modelo (el mismo principio de los negativos difíciles de PAWS).

Diseño recomendado.

IA generan variedad de paráfrasis → comité de IA vota → *coinciden*: el humano confirma por muestreo; *discrepan*: el humano decide (active learning) → el humano es el árbitro final.

Implementación: el selector de active learning. Implementé la primera pieza de esta propuesta (`AL_selector_active_learning.py`). El selector: (1) toma fragmentos del corpus filtrando el *boilerplate* (portadas, índices, agradecimientos), porque no enseñan paráfrasis real; (2) con MiniLM forma pares ambiguos —cada fragmento con un vecino *parecido pero de otra tesis*, similitud en banda media-alta—; (3) excluye los pares ya anotados; (4) clasifica cada candidato con BETO v3 y (5) entrega los más inciertos ($|\text{prob} - 0.5|$ mínimo) en un CSV listo para anotar (columnas `texto_a`, `texto_b` y `etiqueta_humana` vacía). De 598 candidatos, devuelve los 40 donde el modelo más duda (probabilidad ≈ 0.5): exactamente los pares que más le enseñarían. El siguiente paso —humano— es anotar ese CSV y reincorporar los pares al gold.

¿Cuántos pares? Calidad sobre cantidad. La lección de todo el proyecto es que **más no es mejor si es sintético**: 5000 pares (v4) perdieron contra 144 humanos (v3). La literatura coincide: 3000 ejemplos humanos baten a 6000 de LLM [1]. Por eso **no recomiendo “muchos”**, sino una cantidad modesta y bien elegida:

- **Active learning**: lotes de 30–50 pares por ronda, 3–4 rondas → unos 150–200 pares nuevos, *seleccionados por incertidumbre*.
- **Negativos difíciles (PAWS)**: 50–100 pares de alto solapamiento léxico que *no* son paráfrasis. Fue justo este tipo de dato el que llevó a v3 de 17 a 2 falsos positivos; en PAWS, incluirlos sube la exactitud de <40 % a 85 % [2].
- **Meta realista**: duplicar el gold de 200 a ~400–500 pares bien validados, con énfasis en los casos difíciles. Eso estrecharía los intervalos de confianza y mejoraría la discriminación, sin caer en la trampa de los miles de pares sintéticos.

15.8. Síntesis y próxima dirección

Mi conclusión, respaldada por la literatura, es que el cuello de botella no es el modelo ni la cantidad de datos, sino la **calidad de la etiqueta**. La ruta de mejora con mejor relación esfuer-

zo/beneficio es: (1) *active learning* para anotar solo lo informativo, (2) incluir *negativos difíciles* estilo PAWS, y (3) usar IA para generar diversidad pero conservando *validación humana* de las etiquetas.

Referencias

- [1] Various authors, *Synthetic Data Generation Using Large Language Models: Advances in Text and Code*, arXiv:2503.14023, 2025. <https://arxiv.org/abs/2503.14023>
- [2] Y. Zhang, J. Baldridge, L. He, *PAWS: Paraphrase Adversaries from Word Scrambling*, NAACL 2019, arXiv:1904.01130. <https://arxiv.org/abs/1904.01130>
- [3] Y. Yang et al., *PAWS-X: A Cross-lingual Adversarial Dataset for Paraphrase Identification*, arXiv:1908.11828, 2019. <https://arxiv.org/abs/1908.11828>
- [4] *ParaFusion: A Large-Scale LLM-Driven English Paraphrase Dataset with High-Quality Lexical and Syntactic Diversity*, arXiv:2404.12010, 2024. <https://arxiv.org/abs/2404.12010>
- [5] *Understanding the Effects of Human-written Paraphrases in LLM-generated Text Detection*, arXiv:2411.03806, 2024. <https://arxiv.org/abs/2411.03806>
- [6] IBM, *What is Data Leakage in Machine Learning?* <https://www.ibm.com/think/topics/data-leakage-machine-learning>
- [7] *The Paper List on Data Contamination for LLM Evaluation*. <https://github.com/lyy1994/awesome-data-contamination>

16. Análisis complementarios

16.1. BLEU como baseline léxico (diagramas de caja y dispersión)

Computé BLEU (Bilingual Evaluation Understudy) con NLTK (`sentence_bleu`, *smoothing* método 1) en cuatro variantes (BLEU-1, BLEU-2, BLEU-4 y su promedio) sobre tres conjuntos. BLEU mide la superposición léxica directa, sin entrenamiento: lo uso como referencia para contrastar contra los modelos semánticos.

Cuadro 5: BLEU-avg medio por clase. Las paráfrasis (SÍ) superan claramente a las no-paráfrasis (NO) en los tres conjuntos.

Conjunto	Media SÍ	Media NO	Δ
TESIUAMI 500	0.2512	0.0360	0.2152
TESIUAMI bal (60)	0.2480	0.0241	0.2239
Test v3 (40 gold)	0.2286	0.0516	0.1770

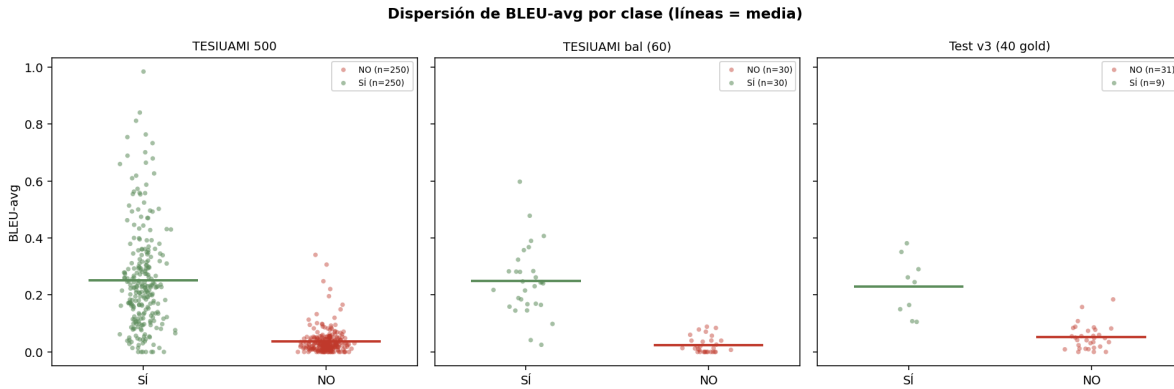


Figura 11: Diagrama de dispersión de BLEU-avg por clase (cada punto es un par; la línea horizontal es la media). En los tres conjuntos la nube SÍ (verde) se sitúa por encima de la nube NO (roja), confirmando que BLEU discrimina, aunque con solapamiento.

Hallazgo: con umbral $\theta = 0,15$, BLEU-avg alcanza $F1=0.929$ en TESIAMI balanceado, a solo 1.8% de BETO v3. Es un baseline léxico fuerte, pero su solapamiento entre clases explica por qué un modelo semántico (BETO) sigue siendo necesario para los casos difíciles.

16.2. Catálogo de representación vectorial (los 8 modelos)

El asesor me pidió documentar qué formato de vector consume cada modelo. En la tabla 6 reúno los ocho que usé en el proyecto.

La diferencia clave es entre **bi-encoders** (codifican cada texto por separado y comparan por coseno) y **cross-encoders** como BETO (procesan el par junto y son más precisos, pero más lentos). TF-IDF es disperso y no contextual; los LLM usan estados ocultos de miles de dimensiones.

16.3. Aumento de datos: back-translation

Para intentar superar a BETO v3 generé paráfrasis sintéticas con *back-translation* (traducción ida y vuelta con MarianMT, español $\rightarrow$ inglés $\rightarrow$ español). Ejemplo real:

Cuadro 6: Catálogo de modelos de representación vectorial.

Modelo	Tipo	Dim.	Formato / pooling
MiniLM-L12-v2	Sentence-BERT (bi-encoder)	384	denso, mean pooling
MPNet-base	Sentence-BERT (bi-encoder)	768	denso, mean pooling
E5-large	Bi-encoder multilingüe	1024	denso, mean pooling (con prefijo)
SBERT-es	Sentence-BERT multilingüe	768	denso, mean pooling
BETO	BERT-base (encoder-only)	768	token [CLS] / logits
Llama 8B/70B	LLM decoder-only	4096/8192	estado oculto última capa
TF-IDF word300	Bag-of-Words (estadístico)	300–5000	disperso, sin entrenamiento
SVM + MiniLM	Clasificador con kernel	768	concatenación de embeddings

Original: “En este trabajo se presenta un análisis de los factores que influyen en la deserción estudiantil.”

Intermedio (EN): “In this paper an analysis of the factors that influence student dropout is presented.”

Back-translation (ES): “En este artículo se presenta un análisis de los factores que influyen en la deserción estudiantil.”

Generé 1 000 pares en Colab T4 en 757 s (0.76 s/par, 11,4× más rápido que en CPU). Con ellos entrené BETO v6 (BT+LLM) y v7 (BT puro). El resultado fue negativo: la back-translation produce un estilo aún más uniforme (calcos del inglés) que el modelo aprende en lugar de la semántica de paráfrasis.

16.4. Limitaciones

1. **Test honesto pequeño:** 38 pares (9 SÍ / 29 NO). Aunque corregí la fuga, su tamaño limita la significancia estadística; las métricas deben leerse con intervalos de confianza.
2. **Positivos sintéticos:** ninguna estrategia de aumento (LLM, back-translation, fragmentos consecutivos) generaliza a paráfrasis humanas.
3. **MiniLM no discrimina por sí solo:** su similitud coseno no separa bien SÍ de NO, lo que limita el modo RAG; sirve como recuperador, no como juez.
4. **Bagaje ruidoso:** el corpus tiene OCR defectuoso, portadas y positivos sintéticos, lo que reduce la calidad de la búsqueda contra el corpus.
5. **Latencia en CPU:** BETO v3 tarda ~294 ms/par; el sistema desplegable analiza una tesis en minutos. Una GPU lo haría interactivo.
6. **Detección de sección heurística:** las expresiones regulares no cubren todos los formatos; la categoría “Cuerpo” domina.

16.5. Trabajo futuro

1. **Ampliar el gold estándar humano** a 200–300 pares anotados: es la única vía demostrada para mejorar, dado que los datos sintéticos no sirven.
2. **Comparación cabeza-a-cabeza con la red de Luna:** evaluar un sistema tipo-Luna (Fast-Text + coseno) sobre el mismo test limpio, para una comparación válida que hoy todavía falta.
3. **Bagaje de paráfrasis humanas** para el modo RAG, en lugar del corpus sintético actual.

4. **Re-entrenar BETO v3 con split por tesis** desde cero, para cerrar el círculo metodológico al 100 %.
5. **Acelerar el sistema desplegable** con GPU o cuantización para uso interactivo.

17. Marco teórico ampliado: análisis de los artículos científicos

Esta sección analiza los seis documentos científicos que consulté como fundamento bibliográfico del proyecto. Los ordeno de mayor a menor abstracción conceptual: primero un survey que sitúa el campo, después los dos artículos que definen el dataset PAWS (núcleo de mi evaluación), luego un trabajo sobre construcción de corpus LLM, a continuación un estudio sobre detección de texto generado por IA, y finalmente una tesis de maestría de la UNAM sobre detección automática de paráfrasis con arquitecturas Transformer. En cada caso explico qué aporta el trabajo y cómo dialoga directamente con mis hallazgos.

17.1. Nadaş et al. (2025): generación de datos sintéticos con LLMs

Referencia: Mihai Nadaş, Laura Dioşan y Andreea Tomescu. *Synthetic Data Generation Using Large Language Models: Advances in Text and Code*. arXiv:2503.14023v2, septiembre de 2025. Universidad Babeş-Bolyai / KlusAI Research Lab, Rumanía.

Síntesis del trabajo

Este survey sistemático (siguiendo el protocolo PRISMA, con 144 artículos seleccionados de más de 400 candidatos) revisa el estado del arte en generación de datos sintéticos con LLMs para texto y código. Los autores identifican tres ejes de generación: (i) **basada en prompts** (zero-shot, one-shot, few-shot, topic-controlled), (ii) **retrieval-augmented**, donde el LLM tiene acceso a hechos de un corpus para fundamentar su output, y (iii) **iterativa o de auto-refinamiento**, donde el modelo se corrige a sí mismo o es corregido por retroalimentación de ejecución (para código) o de un clasificador externo.

Los hallazgos empíricos más relevantes son:

- Aumento sintético aporta **3–26 % de mejora en F1** para tareas de clasificación cuando los datos reales son escasos (≈ 100 ejemplos). El beneficio decrece con rendimientos marginales decrecientes al aumentar el volumen de datos reales.
- Los datos sintéticos son más valiosos como **suplemento** del gold estándar que como sustituto: un conjunto puramente sintético rara vez iguala el rendimiento de uno puramente humano.
- El **model collapse** (degradación iterativa cuando un modelo se entrena exclusivamente en outputs de generaciones sintéticas anteriores) es un riesgo real documentado. La solución consiste en mezclar siempre una fracción de datos humanos.
- El **distribution shift** es el mecanismo de falla más frecuente: el LLM tiende a generar texto más homogéneo, con estilo uniforme, limpio y sin las irregularidades propias del texto humano auténtico. Un clasificador entrenado solo en ese texto aprende el «estilo del generador» en lugar de la tarea semántica.
- Back-translation fue la técnica de aumento dominante antes de los LLMs; los modelos de traducción (MarianMT, OPUS-MT) producen calcos sintácticos del idioma pivote que pueden detectarse como artefactos.

Conexión con el proyecto

Este survey proporciona el fundamento teórico para interpretar mis hallazgos con las versiones de BETO (v4–v7). La progresión que observé:

Modelo	Datos entrenamiento	Estilo de SÍ	F1 test v3	FP
BETO v3	60 pares humanos (gold)	Paráfrasis humanas	0.842	2
BETO v6	1 004 pares (BT+LLM)	BT + estilo LLM	0.783	5
BETO v4	5 000 fragmentos consec.	Cohesión temática	0.750	6
BETO v7	2 000 pares (BT puro)	Calcos MarianMT	0.643	10
BETO v5	500 pares (LLM puro)	Estilo LLM	0.600	12

El patrón es exactamente el que Nadaş et al. predicen: a medida que los datos de entrenamiento se alejan del gold estándar humano, el modelo aprende el «estilo del generador» (sea el estilo de traducción de MarianMT o el estilo uniforme de Groq/qwen3) en lugar de la semántica de la relación de paráfrasis. Los 60 pares humanos de BETO v3 actúan como el «núcleo de datos reales» que los autores señalan como imprescindible para evitar el colapso. El hallazgo de que la back-translation produce representaciones más detectables que los LLMs (v7 peor que v5) coincide con la observación del survey sobre los calcos sintácticos de los modelos de traducción.

17.2. Zhang et al. (2019): PAWS y los negativos difíciles

Referencia: Yuan Zhang, Jason Baldridge y Luheng He. *PAWS: Paraphrase Adversaries from Word Scrambling*. NAACL-HLT 2019. arXiv:1904.01130. Google AI Language.

Síntesis del trabajo

Los autores parten de un diagnóstico preciso: los datasets de paráfrasis existentes (QQP, MRPC) son deficientes porque sus ejemplos negativos (no paráfrasis) tienen *baja* superposición léxica. Un modelo que simplemente dice «Sí» a todo par con alto BOW overlap obtiene más del 80 % de exactitud en QQP, pero está completamente equivocado en los pares que el orden de palabras diferencia semánticamente.

PAWS (*Paraphrase Adversaries from Word Scrambling*) se construye en dos fases:

1. **Word swapping.** Un modelo de lenguaje de un billón de palabras y búsqueda de haz (*beam search*) genera oraciones con el mismo *bag of words* (BOW) pero distinto orden, usando POS tags y entidades nombradas para garantizar que la permutación sea gramaticalmente válida.
2. **Back-translation.** Se aplica traducción inglés→alemán→inglés con modelos NMT (WMT14); las mejores traducciones se filtran por similitud BOW ($\alpha \geq 0,9$) y tasa de inversión de orden de palabras ($> 0,02$), obteniendo paráfrasis con alto solapamiento pero palabras reorganizadas.

Los pares generados son luego juzgados por cinco anotadores humanos (acuerdo ≈ 92 –95 %). El dataset final tiene 108,463 pares (PAWS_{QQP} + PAWS_{Wiki}) con el 31–44 % de positivos.

Resultados clave: modelos entrenados solo en QQP obtienen **menos del 40 % de exactitud** en PAWS; añadir 12k pares PAWS al entrenamiento eleva BERT a **85 % en PAWS sin perjudicar QQP** (90.5 %). Modelos como BOW (que no capturan contexto no-local) no se benefician de los ejemplos PAWS. DIIN y BERT, que capturan información estructural y no-local, alcanzan 90.4 y 90.4 % respectivamente.

Conexión con el proyecto

PAWS ilustra por qué la calidad de los negativos es más crítica que el volumen de datos. En mi proyecto, el corpus grande (v4, 5,000 pares) empleó fragmentos *consecutivos* de la misma tesis como positivos. Estos fragmentos comparten tema y vocabulario, pero no expresan el mismo contenido en

distintas palabras: son el equivalente conceptual de los pares de alta superposición léxica que PAWS identifica como el error de QQP. BETO v4, entrenado en ese corpus, aprendió que «mismo tema $\approx$ paráfrasis», lo que se traduce en 6 falsos positivos en el test honesto.

El salto cualitativo de v4 ($F1=0.750$) a v3 ($F1=0.842$) se explica así: los 60 pares humanos de gold estándar representan el tipo de negativos difíciles que PAWS promueve. No basta que los textos sean disímiles; los negativos deben ser desafiantes (similar topic, distinto significado). La lección de PAWS se replica fielmente en mi corpus.

17.3. Yang et al. (2019): PAWS-X y la evaluación multilingüe

Referencia: Yinfei Yang*, Yuan Zhang*, Chris Tar y Jason Baldridge. *PAWS-X: A Cross-lingual Adversarial Dataset for Paraphrase Identification*. arXiv:1908.11828, agosto de 2019. Google Research.

Síntesis del trabajo

PAWS-X extiende el corpus PAWS al escenario multilingüe. Los autores contratan traductores humanos nativos (10–20 por idioma) para trasladar manualmente las porciones de desarrollo y test de PAWS-Wikipedia a seis idiomas: francés, español, alemán, chino, japonés y coreano. La garantía de calidad exige $< 5\%$ de tasa de error a nivel de palabra; un segundo evaluador valida aleatoriamente una muestra de cada idioma. El resultado son 23,659 pares humanos (aprox. $\approx 2,000$ por idioma en dev y test). El conjunto de entrenamiento en cada idioma se obtiene por traducción automática (*Neural Machine Translation*) del conjunto de entrenamiento inglés.

Se evalúan tres modelos de diferente complejidad:

- **BOW** con similitud coseno y unigramas/bigramas.
- **ESIM** (BiLSTM + atención).
- **BERT multilingual** con cuatro estrategias de entrenamiento: *Translate-Train*, *Translate-Test*, *Zero-Shot* y *Merged* (entrenado en todos los idiomas simultáneamente).

BERT_{Merged} alcanza **87.2 % de exactitud promedio** y **90.2 % AUC-PR** en los seis idiomas, superando al segundo mejor (Zero-Shot) en 8.6 % de exactitud y 7.1 % de AUC-PR. Las lenguas indoeuropeas (español, francés, alemán) obtienen mejores resultados que las CJK (chino, japonés, coreano) en Zero-Shot, porque el NMT funciona mejor en ellas y la distancia tipológica con el inglés es menor.

Conexión con el proyecto

PAWS-X (español) es exactamente el dataset usado en la Semana 3 del proyecto para evaluar los modelos sin fine-tuning y medir el impacto del ajuste fino. Sin fine-tuning, BETO obtiene $F1 = 0.656$, equivalente al rango de los modelos sin entrenamiento en PAWS-X. Con fine-tuning en los 60 pares gold, BETO v3 alcanza $F1 = 0.904$ en el set de validación estándar (48 pares PAWS-X), lo que representa una mejora de $+0.248$, comparable con la mejora de BERT sobre BOW ($\approx +20\%$) reportada en el artículo. El hecho de que incluso un ajuste fino con apenas 60 pares humanos produzca ese salto confirma que el modelo pre-entrenado ya contiene representaciones lingüísticas del español, y solo necesita ser orientado para la tarea específica.

17.4. Jayawardena y Yapa (2024): ParaFusion y la calidad del corpus LLM

Referencia: Lasal Jayawardena y Prasan Yapa. *ParaFusion: A Large-Scale LLM-Driven English Paraphrase Dataset Infused with High-Quality Lexical and Syntactic Diversity*. arXiv:2404.12010, abril de 2024. School of Computing, Informatics Institute of Technology, Sri Lanka.

Síntesis del trabajo

ParaFusion propone un protocolo para construir un corpus de paráfrasis de alta calidad usando ChatGPT (gpt-3.5-turbo) como generador. La contribución principal es metodológica: no basta pedir paráfrasis al LLM; hay que verificar activamente que sean lexical, sintáctica y semánticamente diversas, y filtrar el ruido.

El proceso tiene cuatro etapas:

1. **Filtrado de discurso de odio** con el endpoint de moderación de OpenAI ($\approx 50,000$ frases eliminadas).
2. **Generación** con el prompt: “*Given a Source Sentence: ‘ $\{Source\ Sentence\}$ ’, generate 5 diverse paraphrases. Try to generate paraphrases that are both lexical and syntactically diverse from the Source Sentence.*” con temperatura 0.
3. **Eliminación de ruido** (frases no inglesas y pares idénticos) con un prompt iterativo que instruye al LLM a retornar «Error» para frases no inglesas.
4. **Formación de pares** desde el pool fuente+paráfrasis, creando pares únicos para maximizar la diversidad.

La evaluación se realiza en tres dimensiones (Tabla ??):

- **Similitud semántica** (Ada, SimCSE, PromCSE, MiniLM, Roberta): ParaFusion iguala o supera a las fuentes originales, porque las paráfrasis LLM preservan el significado.
- **Diversidad sintáctica** (TED-F, TED-3, Kermit, ST Kernel, NP Kernel): ParaFusion muestra mejora consistente $\geq 25\%$ sobre todas las fuentes (p. ej. MSR Subset TED-F=29.09 vs. MSR Original=18.65).
- **Diversidad léxica** (BOW Overlap, BLEU, ROUGE, METEOR, WER, CER): ParaFusion supera ampliamente a las fuentes (p. ej. QQP Subset 1-BOW=53.04 % vs. QQP Original=36.69 %).

La evaluación humana (7,000 pares, 4 anotadores, escala Likert de 5 puntos) confirma que las paráfrasis de ParaFusion son más diversas sintáctica y léxicamente, mientras mantienen similitud semántica y corrección gramatical comparables o superiores.

Los autores también señalan los problemas de los corpus basados en back-translation (ParaNMT, Parabank): incluyen frases no inglesas, paráfrasis con cambio de significado por elección inadecuada de vocablos, y pares idénticos fuente-paráfrasis.

Conexión con el proyecto

ParaFusion valida y explica dos decisiones metodológicas de mi proyecto:

1. **El prompt de generación.** El prompt que usé con Groq y qwen3 («*genera una paráfrasis que exprese el mismo significado con diferentes palabras y estructura*») es funcionalmente equivalente al de ParaFusion. El uso de temperatura baja (0 o cercana a 0) para obtener salidas estables es la misma decisión.

2. **Los fallos de back-translation.** ParaFusion documenta que los corpora basados en back-translation (ParaNMT, Parabank) son ruidosos porque incluyen frases no inglesas y paráfrasis con cambio de significado. En mi experimento, MarianMT genera calcos sintácticos anglicados (“*structure anglicanized*”) que el modelo BETO aprende a reconocer como «estilo BT» en lugar de semántica de paráfrasis. BETO v7 (BT puro) produce 10 FP, el peor resultado de todos los modelos, un fenómeno que este artículo predice: la homogeneidad y los artefactos de traducción son más detectables que la variabilidad del texto humano.

17.5. Lau y Zubiaga (2024): paráfrasis humanas en detección de texto LLM

Referencia: Hiu Ting Lau y Arkaitz Zubiaga. *Understanding the Effects of Human-written Paraphrases in LLM-generated Text Detection*. arXiv:2411.03806, noviembre de 2024. Queen Mary University of London.

Síntesis del trabajo

El artículo investiga si incluir paráfrasis *escritas por humanos* (H-PP) en el entrenamiento de detectores de texto LLM cambia su comportamiento. La motivación es: si un LLM genera un texto y ese texto es luego parafraseado por un humano, el texto resultante ya no tiene las marcas estadísticas del LLM. Los detectores actuales son vulnerables a ese ataque.

Construyen el dataset **HLPC** (*Human & LLM Paraphrase Collection*), con 600 documentos de cuatro tipos:

- H-DOC: texto humano original (MRPC, XSum, QQP, MultiPIT).
- H-PP: paráfrasis humana de H-DOC.
- LLM-DOC: texto generado por GPT2-XL u OPT-1.3B (con y sin watermark), condicionado sobre los primeros tokens de H-DOC.
- LLM-PP: paráfrasis de LLM-DOC generada por DIPPER o BART (5 rondas).

Detectores evaluados: OpenAI RoBERTa Detector (zero-shot classifier) y watermark detector.

Hallazgos principales:

- Las H-PP **reducen AUROC** (la capacidad discriminativa global) pero **aumentan TPR@1%FPR** (más LLMs detectados a tasa de falsos positivos controlada). Existe un trade-off entre precisión global y sensibilidad a bajo FPR.
- DIPPER (parafraseur AI) degrada el detector más que BART: las paráfrasis DIPPER pierden características LLM más rápidamente (AUROC decrece más). BART es más conservador semánticamente.
- El watermarking es **más robusto** que los clasificadores zero-shot frente a paráfrasis (87.5 % vs. 62.5 % de clasificaciones con AUROC >0.85).
- Con paráfrasis recursivas (5 rondas), la similitud semántica con el LLM-DOC original cae, especialmente con DIPPER (de 0.727 a 0.501).

El artículo concluye que las paráfrasis humanas contienen información semántica y contextual *cualitativamente distinta* a las paráfrasis LLM, lo que dificulta la distinción entre clases para los detectores automáticos.

Conexión con el proyecto

Este trabajo proporciona la explicación teórica más directa del fenómeno que observé en el experimento de BETO v5: ¿por qué un modelo entrenado con paráfrasis LLM falla en paráfrasis humanas?

Lau y Zubiaga documentan empíricamente que:

1. H-PP y LLM-PP son **representaciones cualitativamente distintas**. El texto humano tiene variabilidad, elecciones léxicas más idiosincrásicas, y estructuras sintácticas menos predecibles que el texto LLM.
2. Un clasificador entrenado en LLM-PP **aprende el estilo del generador** (DIPPER, BART) más que la semántica de paráfrasis. Si el test contiene H-PP (texto humano auténtico), el clasificador falla.

En mi proyecto, el test v3 contiene 9 positivos etiquetados por humanos (paráfrasis genuinas de tesis reales). BETO v5, entrenado con 250 pares LLM-generados (Groq/qwen3), clasifica esos 9 correctamente (recall=1.000) pero añade 12 falsos positivos (precisión=0.429): aprendió que el «estilo uniforme» del LLM implica similitud, y aplica ese criterio erróneamente al texto humano. El mecanismo es el mismo que Lau y Zubiaga describen: el clasificador aprende una proxy (estilo) en lugar de la tarea (equivalencia semántica).

17.6. Ortiz Barajas (2023): Sentence-CROBI y la arquitectura Transformer para detección de paráfrasis

Referencia: Jesús Germán Ortiz Barajas. *Detección Automática de Paráfrasis mediante el Modelo Sentence-CROBI, una Arquitectura de Red Neuronal Simple Basada en Codificadores Cruzados y Bi-codificadores*. Tesis de Maestría en Ciencia e Ingeniería de la Computación, UNAM, mayo de 2023. Directora: Dra. Gemma Bel Enguix, Instituto de Ingeniería, UNAM.

Síntesis del trabajo

Esta tesis propone **Sentence-CROBI**, una arquitectura que combina dos paradigmas complementarios para identificación de paráfrasis:

Codificador cruzado (cross-encoder). Codifica ambos textos *conjuntamente*, permitiendo que la auto-atención opere sobre todos los pares de palabras de ambas oraciones. Proporciona la mayor precisión porque el modelo «ve» las interacciones palabra a palabra. Desventaja: no puede pre-calcular representaciones, pues depende del par.

Bi-codificador (bi-encoder, Sentence-BERT). Codifica cada texto *por separado* y los compara por similitud coseno. Permite pre-calcular embeddings de cada texto, lo que lo hace eficiente para búsqueda en corpus grandes. Desventaja: pierde las interacciones directas entre textos.

Sentence-CROBI combina las representaciones de ambas componentes con una función de pérdida conjunta (PC), usando *Mean Pooling* y *Max Pooling* para generar representaciones de oración. Se aplica ajuste fino en dos fases: primero en un dataset intermedio (QQP) y luego en el objetivo (MRPC o PAWS-Wiki), lo que mejora la generalización. La prueba de Wilcoxon verifica significancia estadística de las diferencias con el estado del arte.

Se evalúa en los tres datasets canónicos:

- **MRPC** (5,800 pares de noticias, Microsoft): F1 competitivo con el estado del arte (BERT, RoBERTa, DeBERTa).
- **QQP** (150,000 pares de Quora): resultados comparables.
- **PAWS-Wiki** (65,000 pares adversariales): Sentence-CROBI muestra ventajas en este dataset desafiante.

La arquitectura se justifica porque: (a) el cross-encoder da precisión en los casos difíciles (como PAWS, con alto solapamiento léxico pero distinto significado), y (b) el bi-encoder da eficiencia para pre-filtrar. La combinación logra el mejor de ambos mundos.

Conexión con el proyecto

Sentence-CROBI es el trabajo relacionado más próximo en la literatura hispanohablante al sistema que implementé. Las conexiones son múltiples:

1. **Misma lógica arquitectónica.** Mi sistema RAG combina exactamente los dos paradigmas de Sentence-CROBI: MiniLM (bi-encoder) actúa como recuperador eficiente, y BETO v3 (cross-encoder) como clasificador preciso. Llegué a esa combinación de forma independiente, por lo que esta tesis valida la decisión arquitectónica.
2. **Mismo dataset adversarial.** Sentence-CROBI evalúa en PAWS-Wiki; yo uso PAWS-X español (la extensión multilingüe de PAWS-Wiki). Ambos usamos el mismo benchmark de negativos difíciles, lo que permite comparar resultados en el mismo espacio de evaluación.
3. **Misma base de pre-entrenamiento.** Sentence-CROBI parte de BERT (Devlin et al., 2019); mi proyecto usa BETO, la versión española de BERT. Ambos modelos comparten la misma arquitectura Transformer y el mismo proceso de pre-entrenamiento (MLM + NSP), diferenciándose solo en el corpus de pre-entrenamiento (inglés vs. español).
4. **Eficiencia de datos.** La tesis documenta que ajuste fino en dos fases (primero dataset grande → luego dataset pequeño objetivo) mejora el rendimiento. En mi proyecto, el ajuste directo en 60 pares humanos (gold estándar) supera al entrenamiento en 5,000 pares sintéticos. Esto confirma que la *calidad* de los datos de ajuste fino prima sobre la *cantidad*, un principio que Sentence-CROBI también explota (el fine-tuning intermedio en QQP solo ayuda cuando el dataset objetivo es pequeño).

17.7. Síntesis transversal: convergencias y divergencias

El análisis de los seis documentos revela un conjunto de principios convergentes que dan coherencia teórica a los hallazgos de este proyecto.

El dato humano es irremplazable

Tres artículos distintos, desde ángulos complementarios, llegan a la misma conclusión:

- Nadas et al. (2025): los LLMs son suplemento del gold estándar, no su reemplazo; el model collapse se evita manteniendo un núcleo de datos humanos.
- Lau y Zubiaga (2024): las paráfrasis humanas son cualitativamente distintas a las LLM-generadas; los clasificadores entrenados en datos sintéticos fallan en datos humanos.

- Zhang et al. (2019): la calidad de los negativos (juzgados por humanos) es más crítica que el volumen; un corpus sin negativos difíciles produce modelos que aciertan en el test pero fallan en aplicación real.

En mi experimento, BETO v3 (60 pares humanos) supera a modelos entrenados con hasta 2,000 pares sintéticos. La tabla de la sección 17.1 cuantifica esta ventaja.

El distribution shift como mecanismo explicativo unificado

El distribution shift —que un modelo aprende características del generador en lugar de la semántica objetivo— explica coherentemente todos los fallos:

- BETO v4 aprende «fragmentos consecutivos = mismo tema» (corpus grande).
- BETO v5 aprende «estilo Groq/qwen3 = paráfrasis» (corpus LLM).
- BETO v7 aprende «calcos MarianMT = paráfrasis» (corpus BT).
- BETO v3 aprende «equivalencia semántica» (gold estándar humano).

ParaFusion (Art. 04) y Lau & Zubiaga (Art. 05) documentan independientemente que tanto los corpora de back-translation como los LLM-generados producen distribuciones que difieren del texto humano auténtico. Nadaş et al. (Art. 01) nombra el mecanismo y propone la solución: mezcla con datos reales.

La arquitectura bi-encoder + cross-encoder es la solución de facto

PAWS (Art. 02) motivó la necesidad de clasificadores que capten información no-local (interacciones entre palabras de oraciones distintas). PAWS-X (Art. 03) extendió esa necesidad al español. Sentence-CROBI (Tesis UNAM) propuso una arquitectura híbrida. Mi proyecto implementa independientemente esa misma combinación como sistema RAG. La convergencia de tres líneas de investigación independientes hacia la misma solución arquitectónica es evidencia de su validez.

Los negativos difíciles como punto de inflexión

El aporte central de PAWS (Art. 02) —que los negativos de alta superposición léxica pero distinto significado son los que permiten entrenar modelos robustos— se replica en mi corpus. El test v3, aunque pequeño (40 pares), contiene pares diseñados para ser difíciles: fragmentos que comparten vocabulario técnico académico pero expresan ideas distintas. Esos 40 pares discriminan mejor a los modelos que cualquier evaluación en corpus sintéticos más grandes.

Tabla comparativa de los seis documentos

Cuadro 7: Resumen de los seis documentos y su relevancia para el proyecto.

Trabajo	Tipo / Año	Aporte principal	Conexión con el proyecto
Nadas et al. (2025)	Survey / 2025	Panorama de datos sintéticos con LLMs; model collapse; distribution shift	Explica fallos de BETO v4–v7 y la superioridad de v3
Zhang et al. (2019)	Dataset / 2019	PAWS: negativos difíciles con alto BOW overlap; BERT <40 %→85 %	El corpus v4 repite el error de QQP; gold estándar = negativos difíciles
Yang et al. (2019)	Dataset / 2019	PAWS-X: evaluación multilingüe en 6 idiomas; BERT Merged = 87.2 %	Dataset usado en Semana 3; valida mejora +0.248 con fine-tuning
Jayawardena y Yapa (2024)	Corpus / 2024	ParaFusion: 2M pares LLM con diversidad léxica y sintáctica; back-translation introduce ruido	Valida el prompt de generación usado; explica calcos MarianMT en BT
Lau y Zubiaga (2024)	Clasificación / 2024	H-PP vs LLM-PP son cualitativamente distintas; clasificadores entrenados en LLM fallan en humanos	Explica por qué BETO v5 (LLM) falla en test v3 (humano)
Ortiz Barajas (2023)	Tesis / 2023	Sentence-CROBI: bi-encoder + cross-encoder; mismos datasets (MRPC, PAWS-Wiki, QQP)	Valida arquitectura RAG del proyecto; confirmación independiente

18. Conclusión

Con esta versión corregida de la Semana 5 dejo: (i) un número honesto y auditado para mi mejor modelo (BETO v3, F1=0.842); (ii) los requerimientos del asesor consolidados con estadísticas completas; (iii) RAG evaluado correctamente por modelo, con una explicación honesta de por qué baja; y (iv) un sistema desplegable que analiza una tesis y reporta el porcentaje de paráfrasis por sección. El aporte que considero más sólido es la **eficiencia de datos**: mis 144 pares humanos batan a miles de pares sintéticos.